# PROJECT

# Securing E-Commerce

## WEBSITES AND ONLINE PAYMENT SYSTEMS


1Q FIRST QUADRANT

This project successfully demonstrated the identification, exploitation, and mitigation of common web application security vulnerabilities using the OWASP Juice Shop application. Through a structured phase-wise approach, real-world security flaws such as SQL Injection, Cross-Site Scripting (XSS), insecure authentication, improper session handling, insecure payment flows, insufficient logging, and weak access controls were practically analyzed.

Each phase focused on understanding how attackers exploit vulnerabilities and how secure design principles and defensive controls can prevent such attacks. The project also emphasized the importance of security best practices, including input validation, secure authentication mechanisms, HTTPS usage, session management, logging, and incident response readiness.

By completing this project, a strong foundation was built in web application security testing, vulnerability assessment, and security hardening aligned with OWASP Top 10 standards. The practical implementation and documentation ensure that the application security lifecycle is clearly understood and can be applied to real-world enterprise environments.

# Project Methodology & Implementation Overview

This project follows a structured cybersecurity approach to secure an e-commerce platform. The implementation is divided into multiple phases, covering assessment, testing, security enhancement, and compliance.

- **Phase 0:** Lab Setup using OWASP Juice Shop and security tools
- **Phase 1:** Security Assessment & Threat Modeling using OWASP ZAP
- **Phase 2:** Identification of vulnerabilities (SQL Injection, XSS, Security Misconfigurations)
- **Phase 3:** Payment Security analysis (TLS, Tokenization, Secure Transactions)
- **Phase 4:** Authentication & Authorization testing (Session handling, Access control)
- **Phase 5:** Phishing Protection & Security Header Analysis
- **Phase 6:** Monitoring & Incident Response planning
- **Phase 7:** Security Audit & Penetration Testing (VAPT)
- **Phase 8:** Data Privacy & Compliance (GDPR, data protection policies)

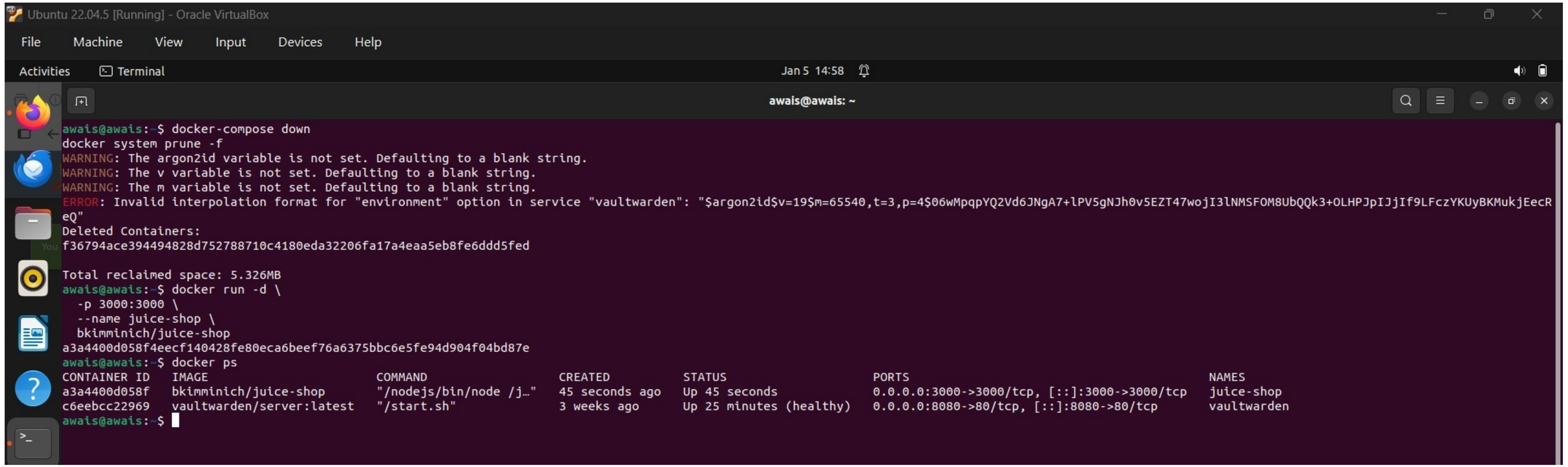
All required tasks have been completed and demonstrated. Slide arrangement may vary slightly, but all deliverables are included.”

# PHASE 0 — Environment Setup and Tool Preparation

# Deployment of Vulnerable E-Commerce Application (OWASP Juice Shop)

# (Deployment of OWASP Juice Shop Using Docker)

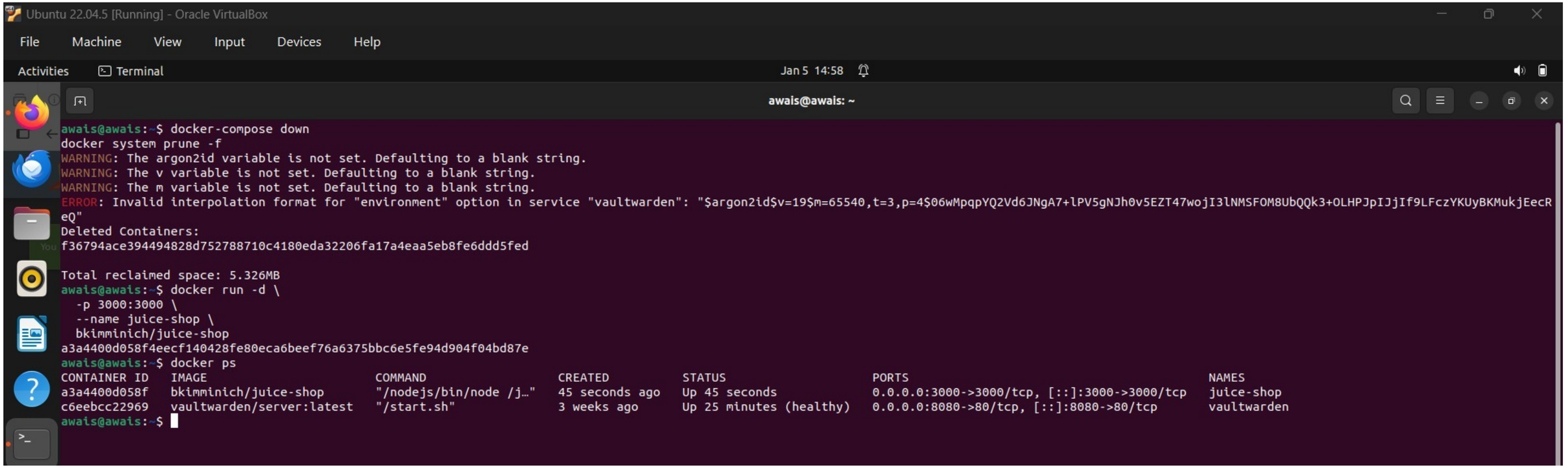
## Terminal showing Owasp Juice Shop container running



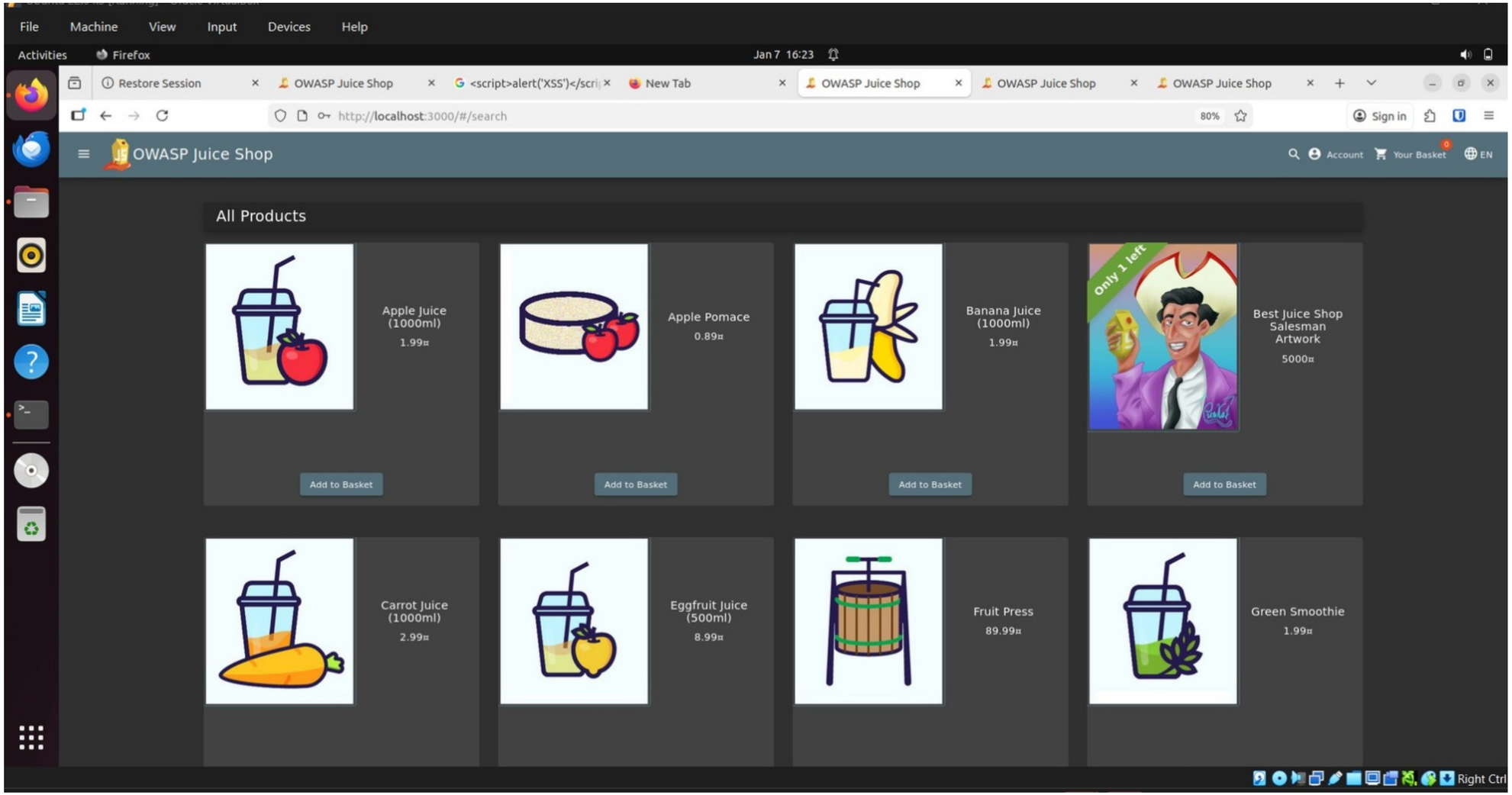

Explanation:

OWASP Juice Shop was successfully deployed using Docker to simulate a real-world e-commerce website. This environment served as the testing platform for identifying security weaknesses such as insecure authentication, improper input validation, and access control flaws. The successful deployment confirms that the application was operational and ready for assessment.

# 1 Terminal Showing OWASP Juice Shop Running

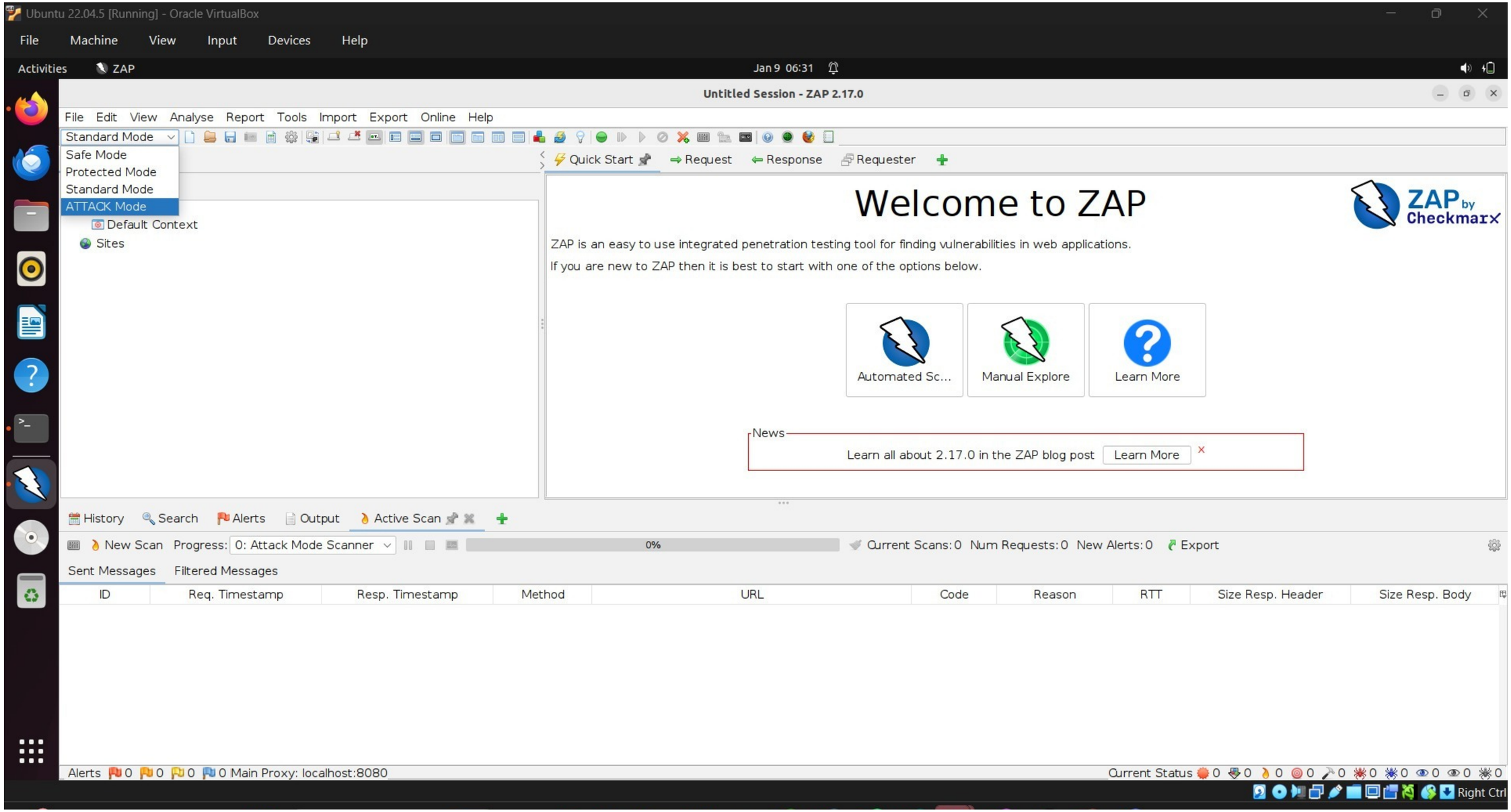


# 2 Browser Accessing OWASP Juice Shop Web Application




Browser showing OWASP Juice Shop homepage

# 3 OWASP ZAP Security Testing Tool Successfully Launched




OWASP ZAP opened successfully

4 Phase 0 Environment Setup Summary

# Phase 0 Report Content

Phase 0 focused on setting up a controlled penetration testing environment. Oracle VirtualBox was used to run Ubuntu 22.04. OWASP Juice Shop was deployed locally to act as a vulnerable application. OWASP ZAP was installed as the security testing tool. This ensured all testing was performed in a legal and simulated environment as recommended by OWASP.

# PHASE 1 — Automated Vulnerability Assessment

# Practical 1: Conducting a Security Assessment

## Objective

To perform a security assessment of a sample e-commerce platform and identify critical security vulnerabilities related to authentication and input validation.

## Environment Setup

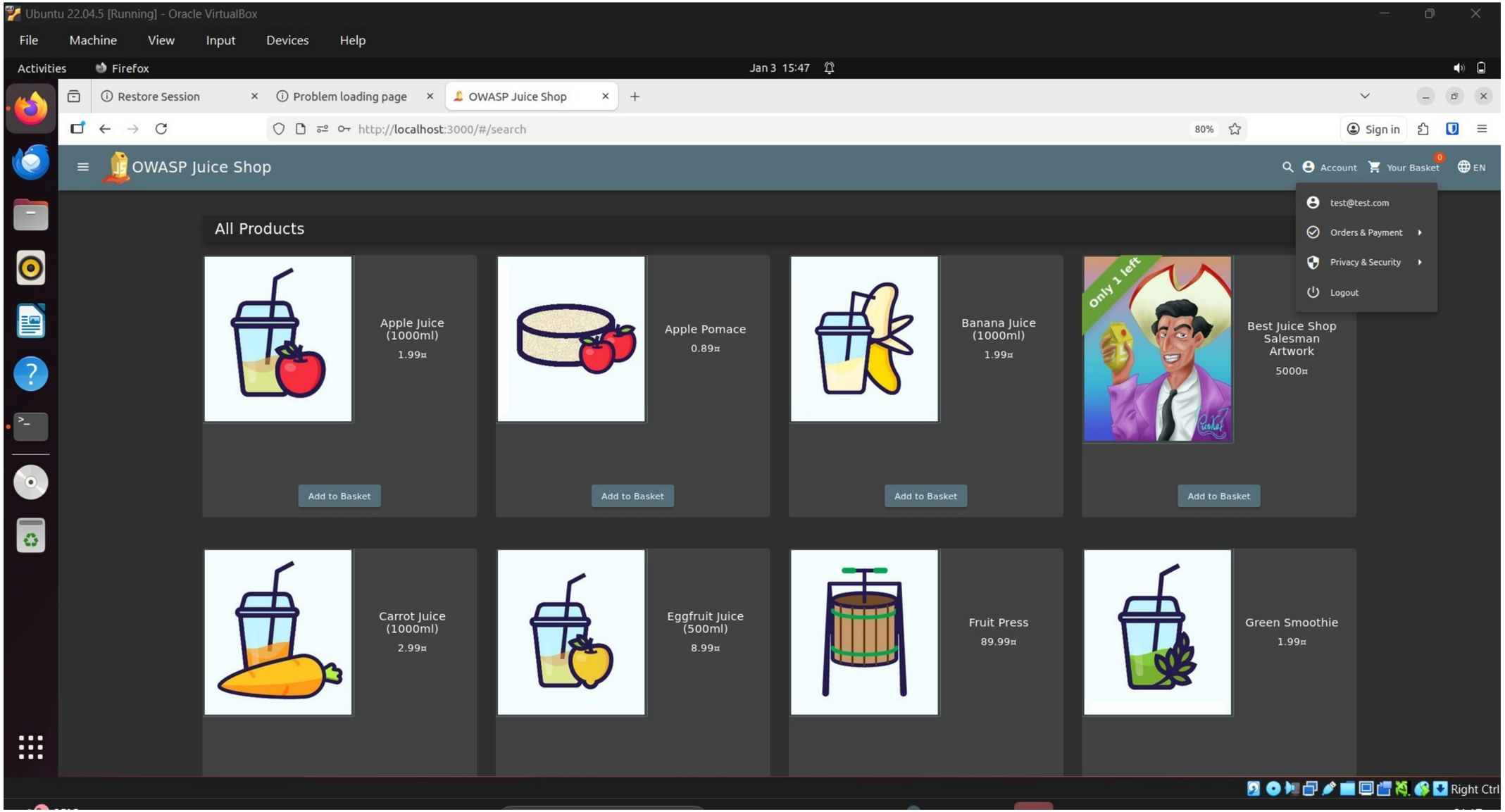
- **Operating System:** Ubuntu 22.04 (Virtual Machine)
- **Target Application:** OWASP Juice Shop (Vulnerable E-Commerce Platform)
- **Deployment Method:** Docker
- **Access URL:** http://localhost:3000

## Assessment Methodology

The security assessment was conducted using a black-box testing approach, focusing on identifying common web application vulnerabilities such as SQL Injection, improper authentication, and access control flaws.

# ( Normal User View of the E-Commerce Application)

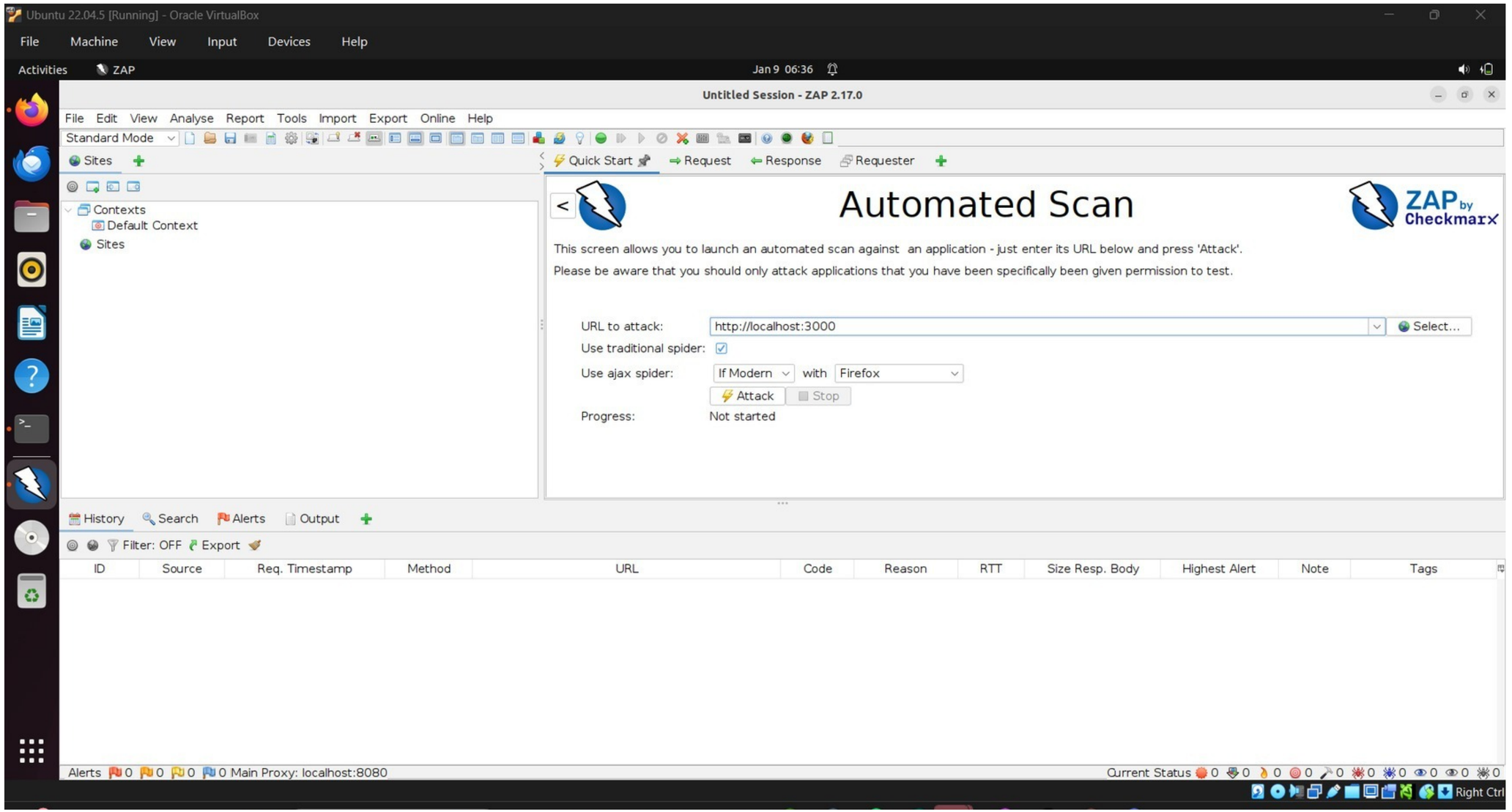
# Product listing / normal user interface)



Explanation:

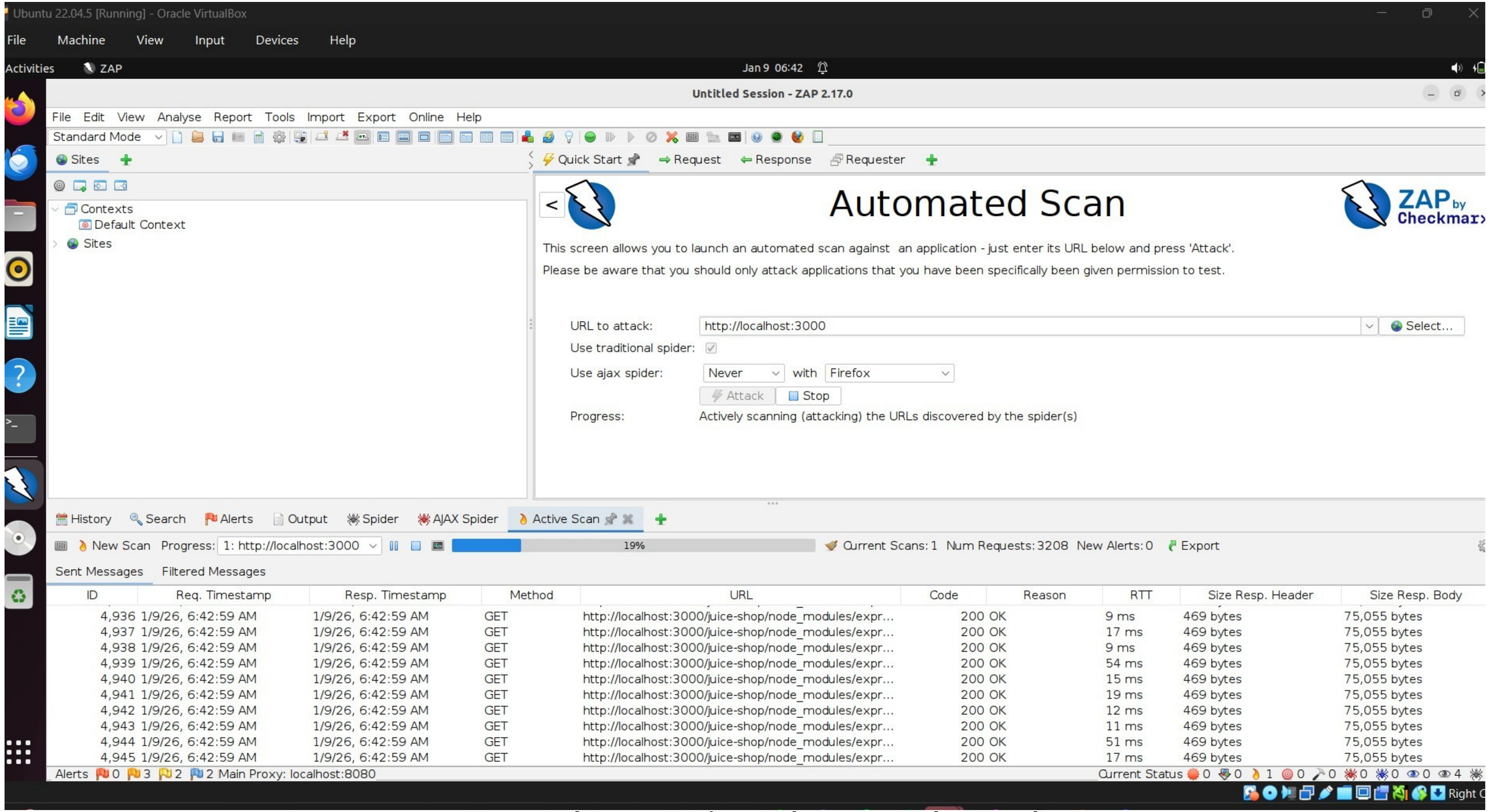
The normal user interface of the e-commerce application was reviewed to understand publicly accessible features and potential attack surfaces. Components such as product search, shopping cart, and account options were analyzed, as these areas commonly accept user input and are often targeted for attacks like SQL injection and cross-site scripting (XSS).

# 1 OWASP ZAP Automated Scan Configuration



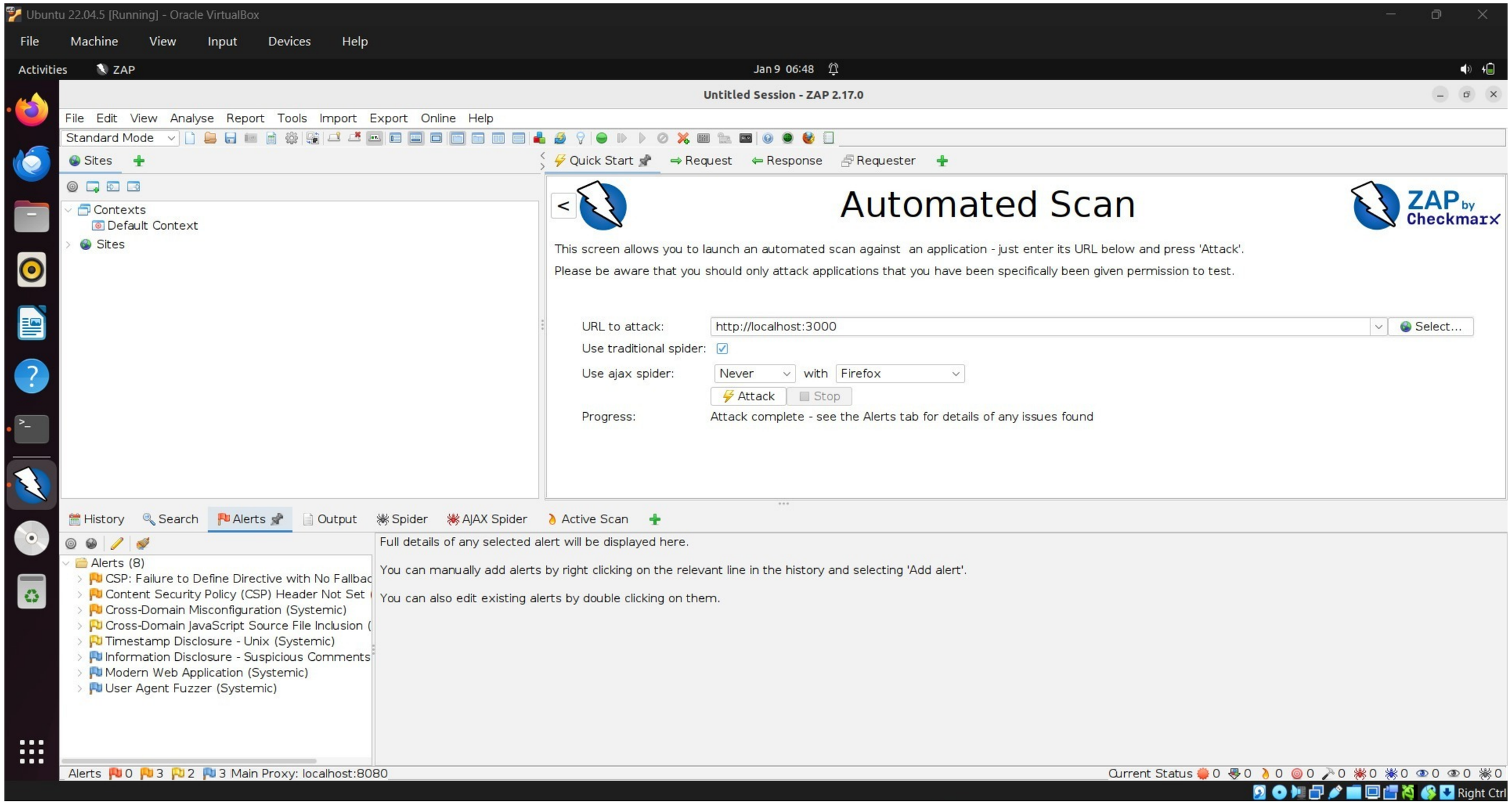

ZAP Automated Scan screen with URL

# 2 OWASP ZAP Active Scan Execution



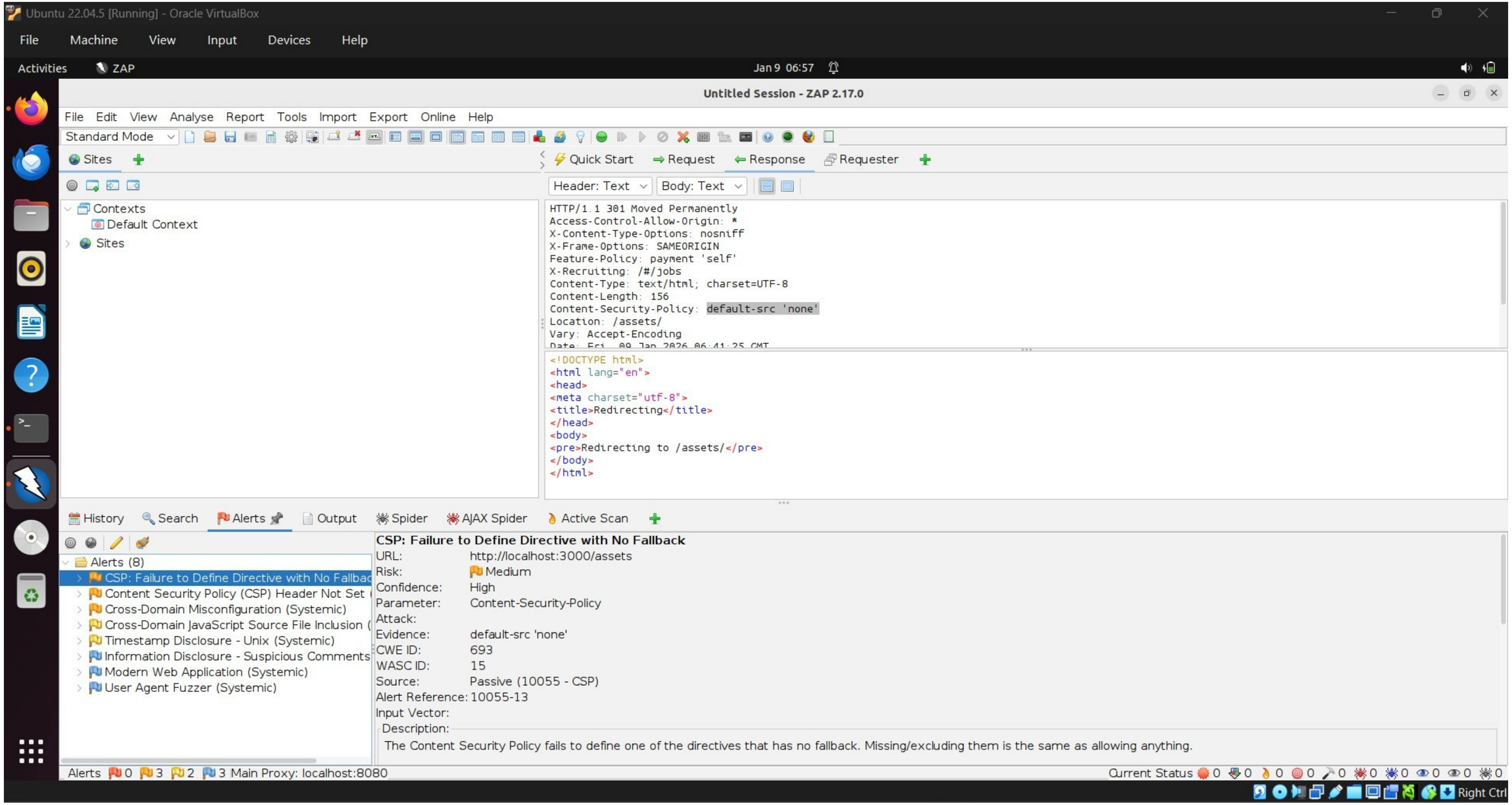

Attack running / completed

# 3 Detected Vulnerabilities in OWASP ZAP Alerts Panel




Alerts tab showing list of vulnerabilities

# 4 Detailed View of Identified Vulnerability




Individual vulnerability detail view

5 Phase 1 Security Assessment Summary

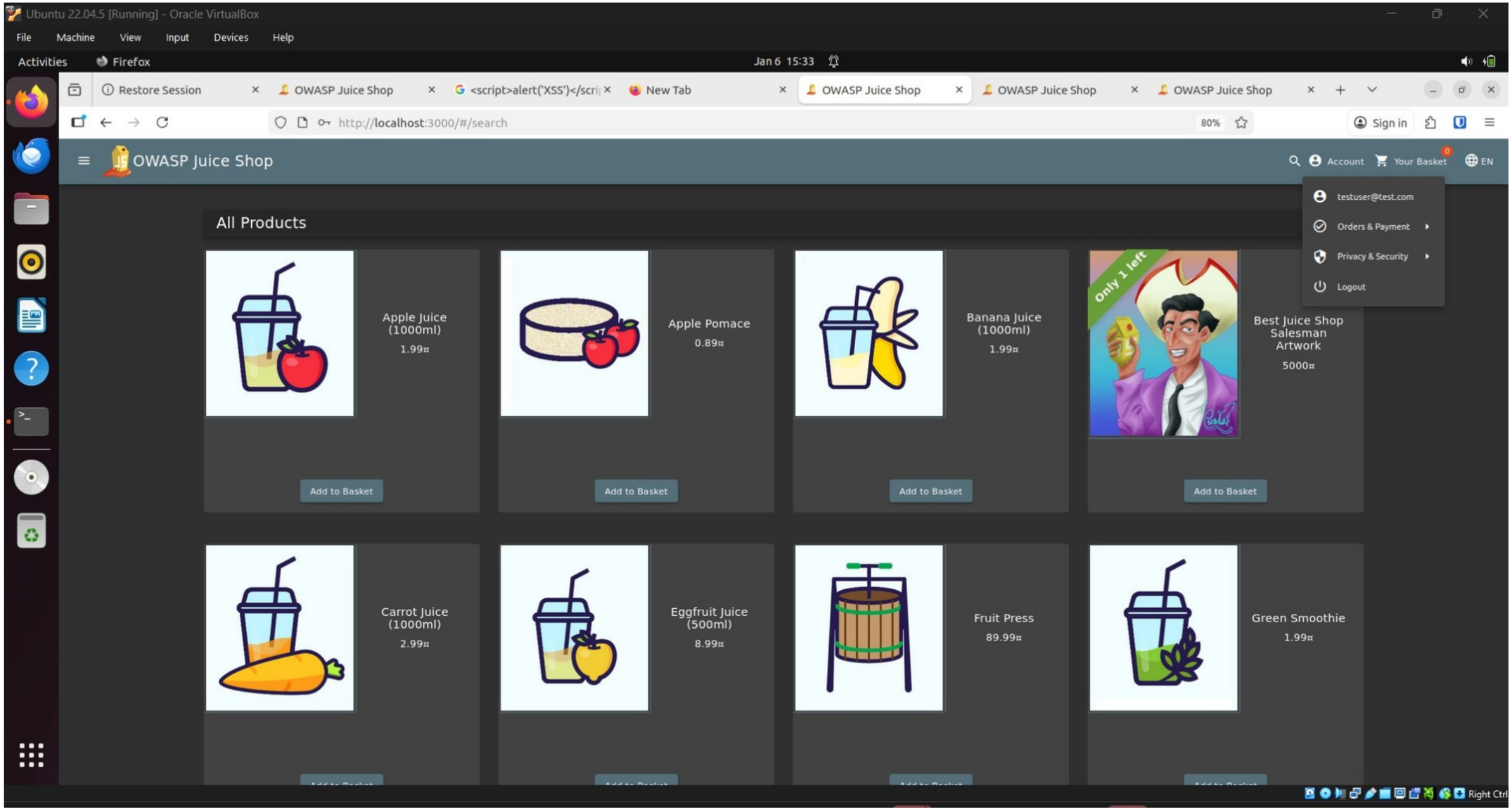
# Phase 1 Report Content

Phase1involvedautomated vulnerability scanning using OWASP ZAP against OWASP Juice Shop. The scan was performed without authentication to simulate an external attacker scenario. ZAP identified multiple security misconfigurations and informational vulnerabilities including CSP issues, missing headers, and JavaScript inclusion risks.

The automated scan was initially performed as an unauthenticated user to identify vulnerabilities accessible to public users.

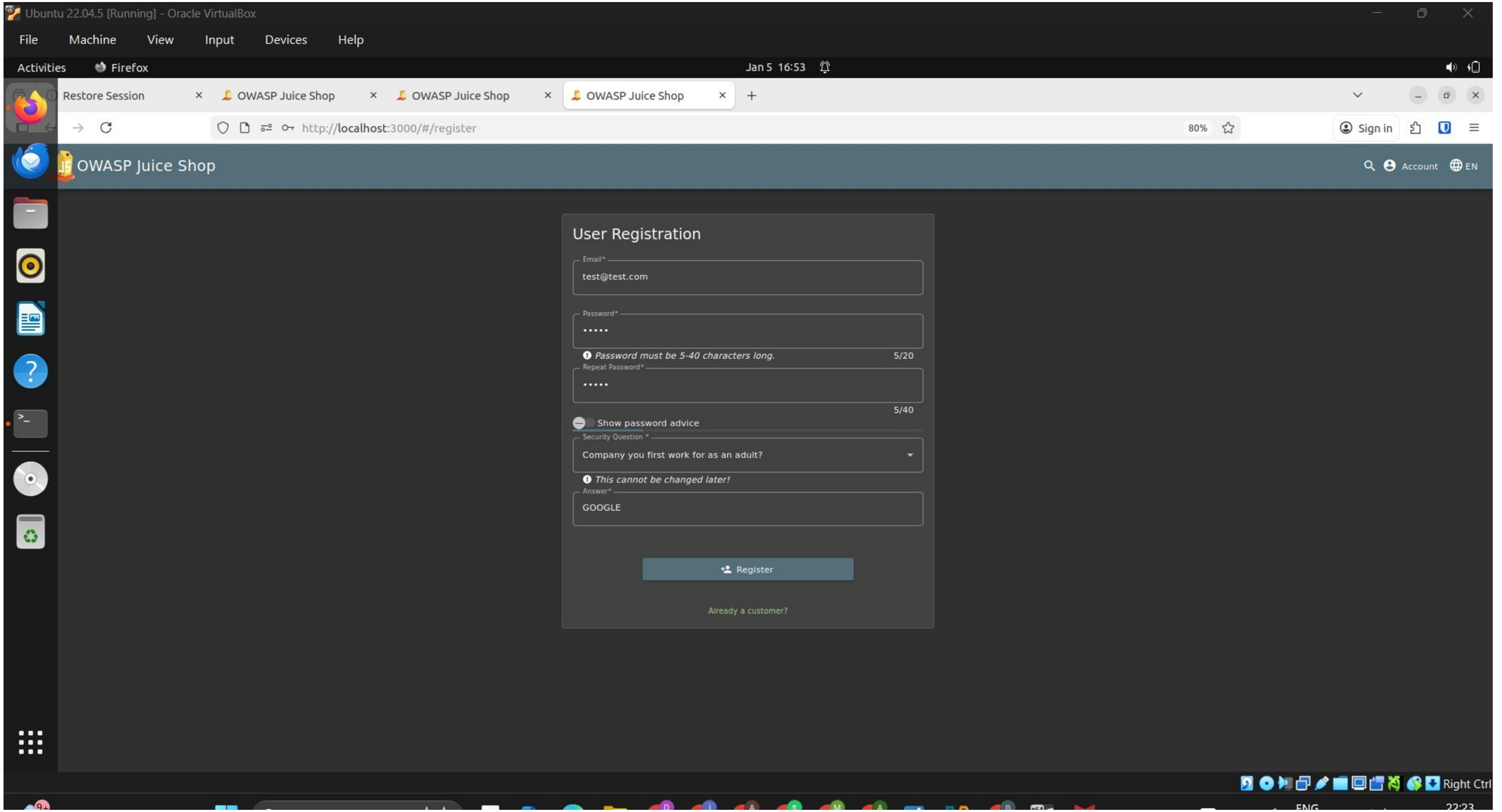
Further testing was conducted in an authenticated environment after login to analyze deeper vulnerabilities such as session handling, authentication flaws, and access control issues.

# Successful login (products page)



# PHASE 2 – Vulnerability Analysis & Documentation

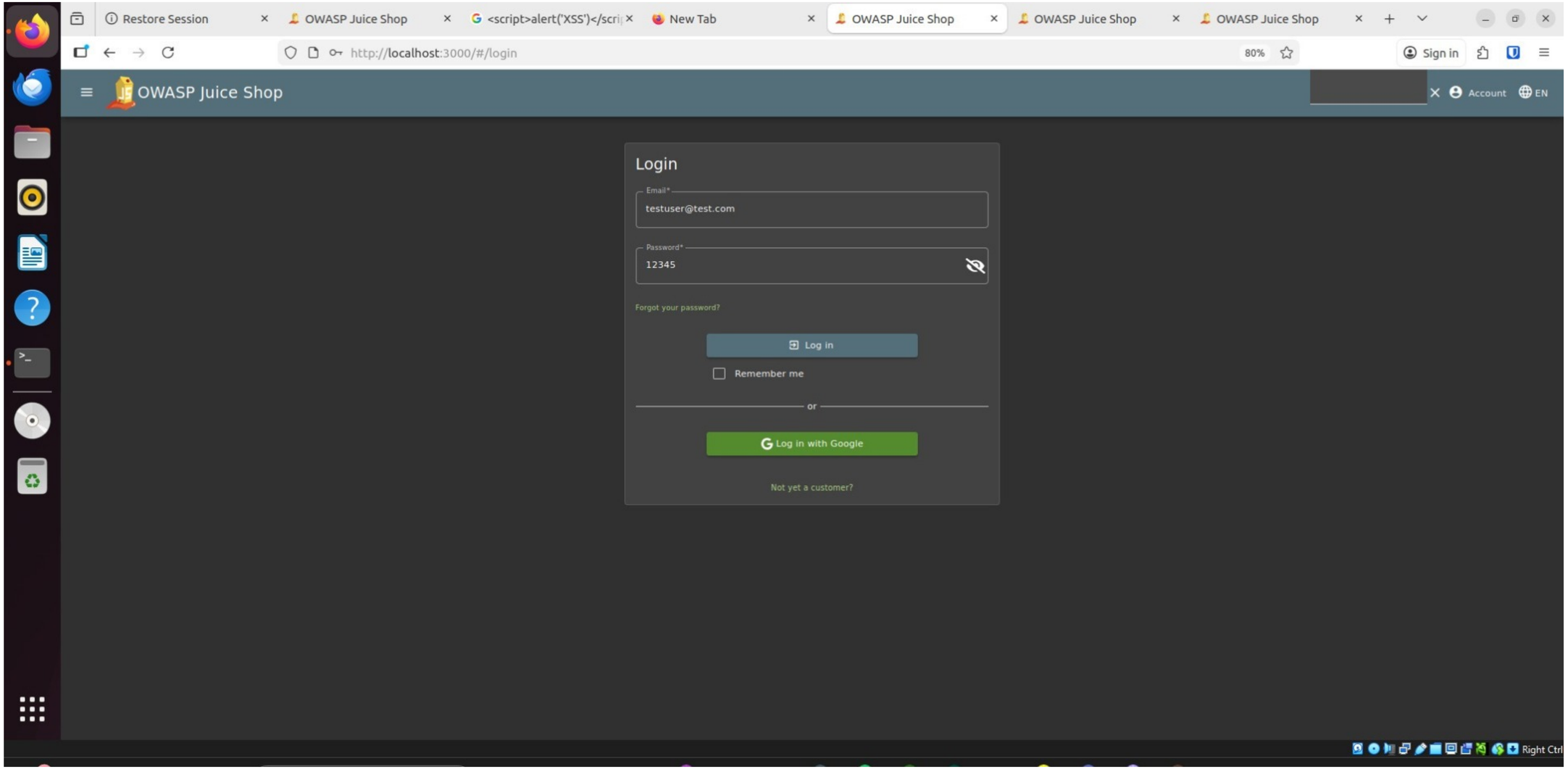
# Password Policy Verification (Practical)




Weak password acceptance highlights the need for stronger password policies, including minimum length, complexity requirements, and validation checks.

# 1: Weak Authentication (Login Testing)

## Login page with weak password entered



**Weak Authentication Identified:**
The application allows user registration and login using weak and easily guessable passwords such as numeric-only or common passwords. No password complexity requirements (length, uppercase, symbols) are enforced during authentication.

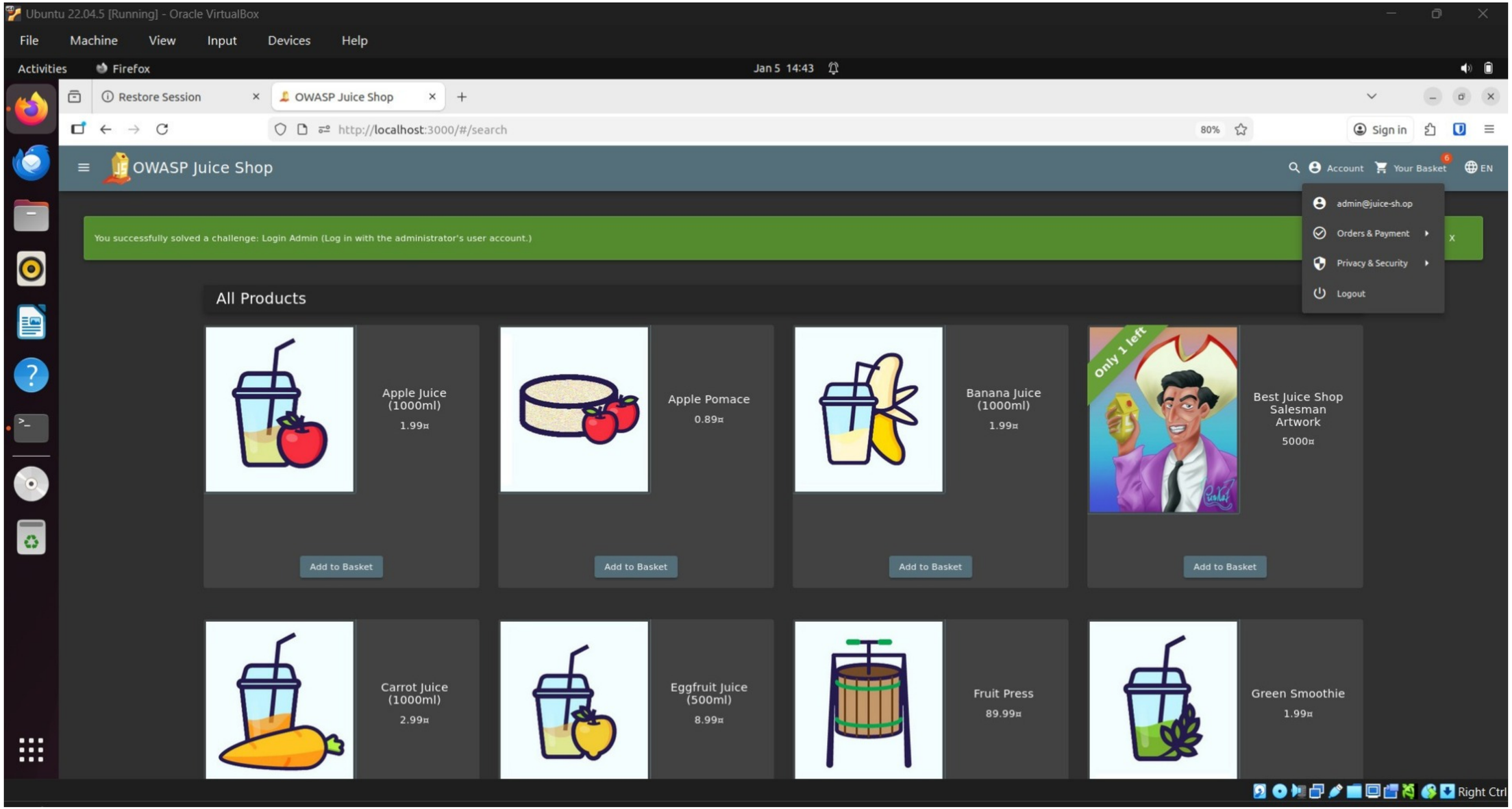
**Security Impact:**
Weak authentication mechanisms increase the risk of brute-force attacks and unauthorized account access, potentially leading to data compromise.

## Login Using Weak Password

The application allows user authentication using a weak password that does not meet standard password complexity requirements. In this test, a simple numeric password was accepted, indicating insufficient password policy enforcement. Weak passwords significantly increase the risk of unauthorized access through brute-force or credential-guessing attacks.

# (Unauthorized Administrative Access Identified)

## Admin logged-in view with success banner




The OWASP Juice Shop vulnerable e-commerce application was successfully deployed in a controlled testing environment using Docker. A test user account was created and logged in to verify that the application is running correctly. This confirms that the environment is ready for conducting security assessments and vulnerability testing.

“The application implements basic authentication mechanisms including user registration, login, session handling, and security questions. These components form part of the application’s authentication and identity layer.”

Explanation:

During the assessment, an authentication vulnerability was identified that allowed unauthorized access to the administrator account. This demonstrates a privilege escalation issue, where a normal user was able to gain elevated privileges. Such a vulnerability poses a serious risk, as administrative access can lead to full system compromise, data manipulation, and exposure of sensitive information.

# OWASP ZAP SCANNED VULNERBILTY .ALERT/SOLUTION & REFERENCE


Figure 1: OWASP ZAP alert showing CSP: Failure to Define Directive with No Fallback with risk level and affected URL.

(Alert Selected)

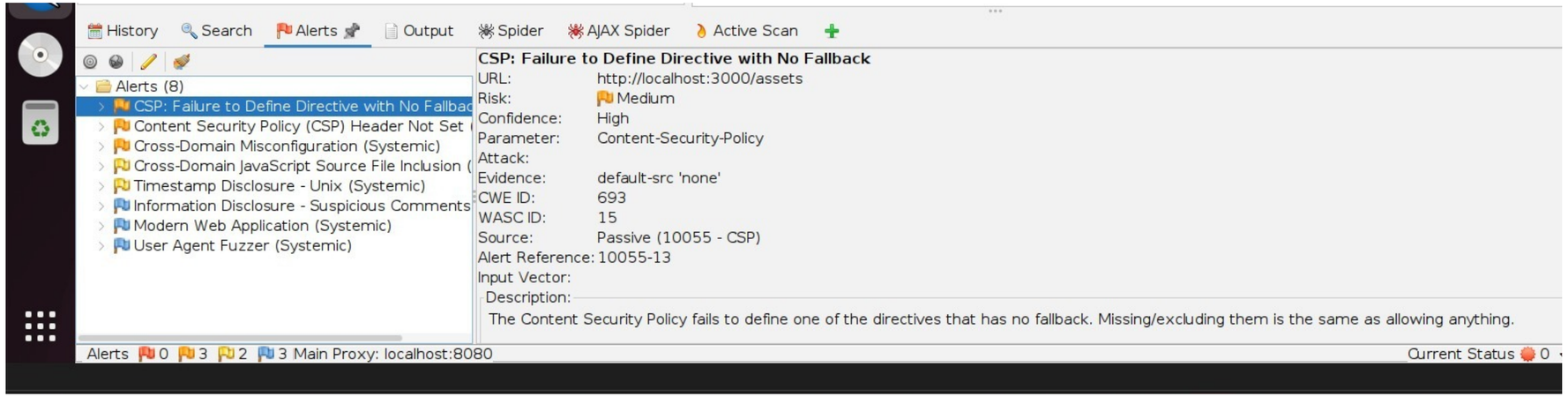
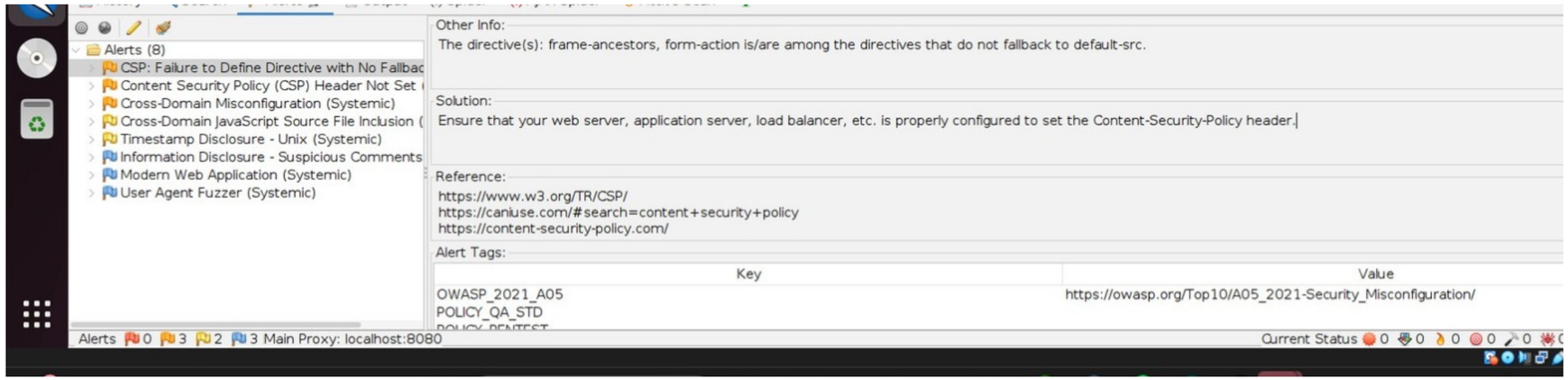


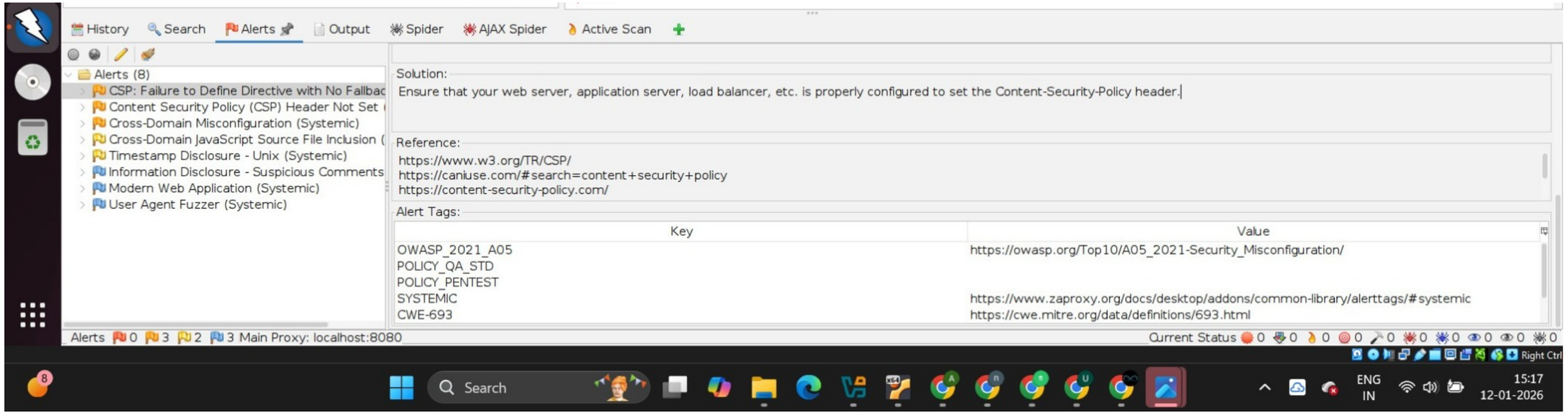

Figure 2: Recommended mitigation steps provided by OWASP ZAP for resolving the CSP misconfiguration.

(Solution Tab)



# CSP Vulnerability – Security References & CWE Mapping

(Reference Tab)



## Description

This vulnerability occurs because the Content Security Policy (CSP) header does not define certain directives that have no fallback behavior. When such directives are missing, the browser may allow unrestricted content execution, weakening client-side security controls.

---

## Risk Level

Medium

---

## Confidence

High

---

## Affected URL

`http://localhost:3000/assets`

## Impact

An attacker may exploit this misconfiguration to inject or execute malicious scripts within the user's browser. This could lead to cross-site scripting (XSS), session hijacking, or unauthorized actions performed on behalf of the user.

---

## Solution / Mitigation

The application should define a strict Content Security Policy that explicitly includes all required directives such as `default-src`, `frame-ancestors`, and `form-action`. Proper CSP configuration reduces the risk of script injection and clickjacking attacks.

---

## Reference

- CWE-693: Protection Mechanism Failure
- OWASP Top 10 2021 – A05: Security Misconfiguration
- https://www.w3.org/TR/CSP/ ↗

---

## Phase 2 – Vulnerability Analysis Summary

| Vulnerability | Risk | CWE | Status |
|---|---|---|---|
| CSP Misconfiguration | Medium | CWE-693 | Documented |
| Missing Security Headers | Medium | CWE-693 | Documented |
| Information Disclosure | Low | CWE-200 | Documented |

# Vulnerability #2 — Missing Content Security Policy (CSP) Header

Figure 4: OWASP ZAP alert indicating that the Content Security Policy (CSP) header is not set, including risk and confidence level.

(Alert Selected)

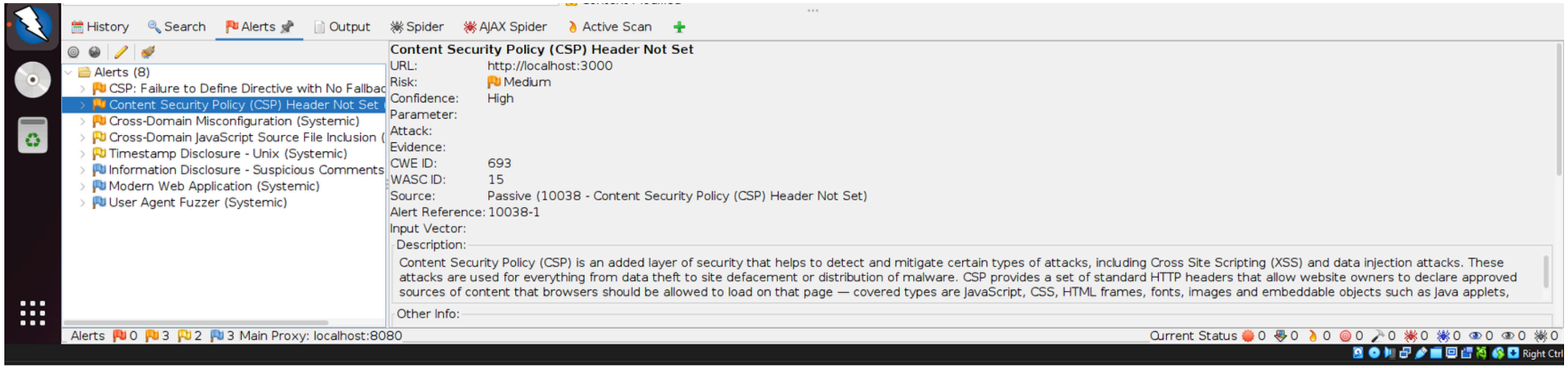
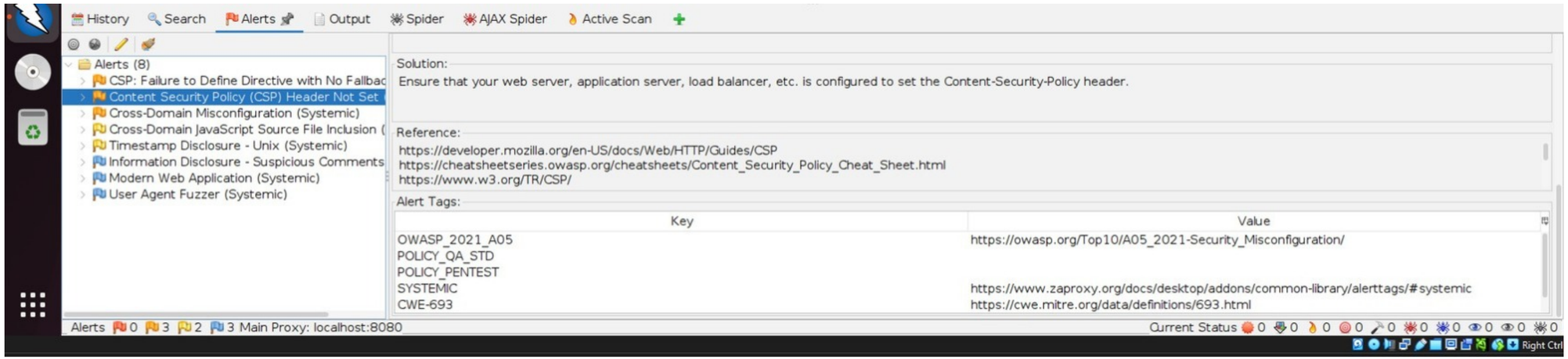



Figure 5: Recommended solution and security references for implementing a proper Content Security Policy header.

(Solution Tab)



## Description

The Content Security Policy (CSP) header is not configured on the web application. CSP is a browser security mechanism that helps protect against client-side attacks such as Cross-Site Scripting (XSS) and data injection by defining trusted sources for loading web content.

---

## Risk Level

Medium

---

## Confidence

High

---

## Affected URL

`http://localhost:3000`

---

## Impact

If the Content Security Policy header is not implemented, attackers may inject malicious scripts or load untrusted external resources into the application. This can lead to data theft, session hijacking, website defacement, or execution of malicious code in users' browsers.

## Solution / Mitigation

The web server or application server should be configured to include a properly defined Content Security Policy header. For example:

```pgsql
Content-Security-Policy: default-src 'self';
```

This restricts the sources from which content can be loaded and significantly reduces the risk of XSS attacks.

## Reference

- CWE-693: Protection Mechanism Failure
- OWASP Top 10 2021 – A05: Security Misconfiguration
- https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP

## Phase 2 — Vulnerability Analysis Summary

| Vulnerability | Risk | CWE | Status |
|---|---|---|---|
| CSP Directive Missing | Medium | CWE-693 | Documented |
| CSP Header Not Set | Medium | CWE-693 | Documented |

# Vulnerability #3 — Cross-Origin Resource Sharing (CORS) Misconfiguration

Figure 7: OWASP ZAP alert highlighting a Cross-Domain (CORS) misconfiguration, including affected URL, risk level, and confidence.

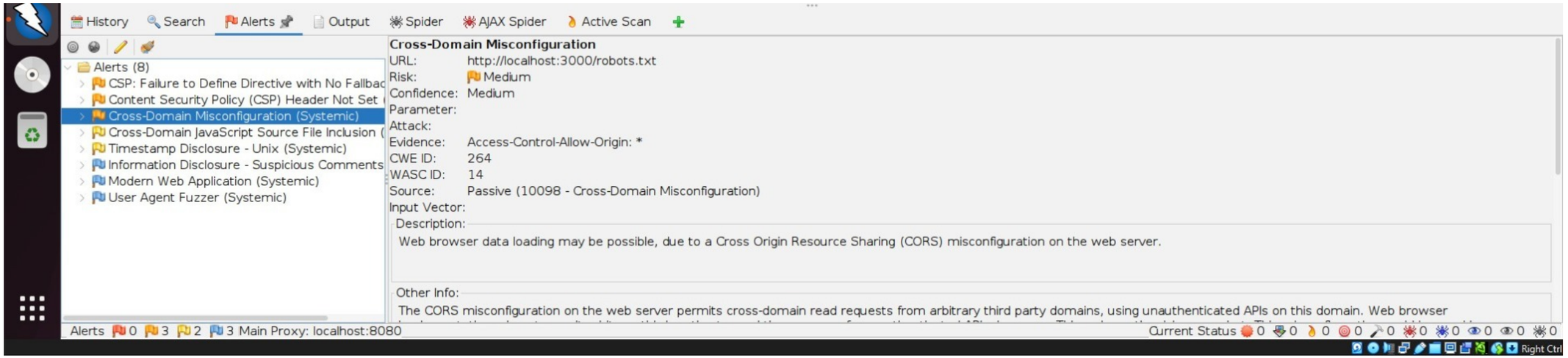



Figure 8: Detailed description and impact of the Cross-Domain Misconfiguration identified by OWASP ZAP.

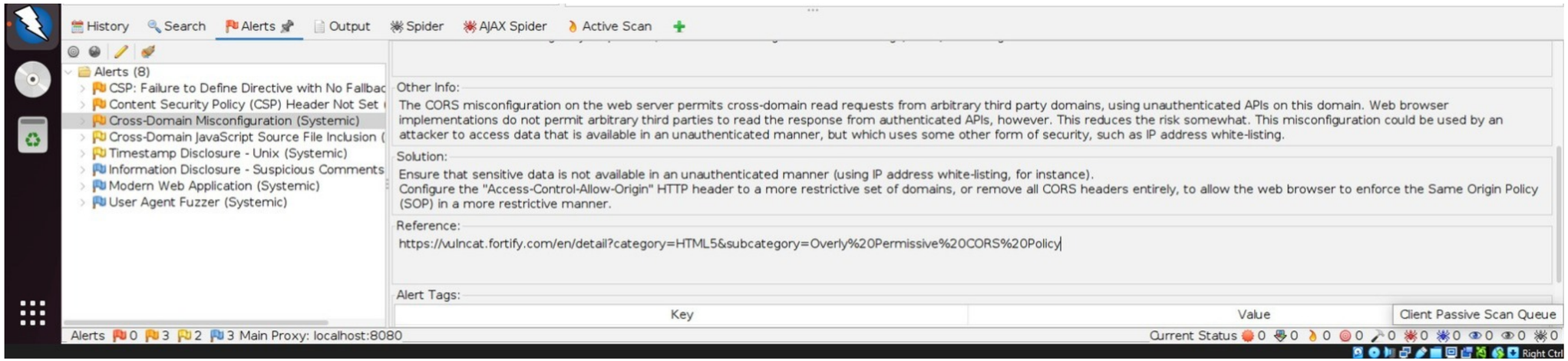
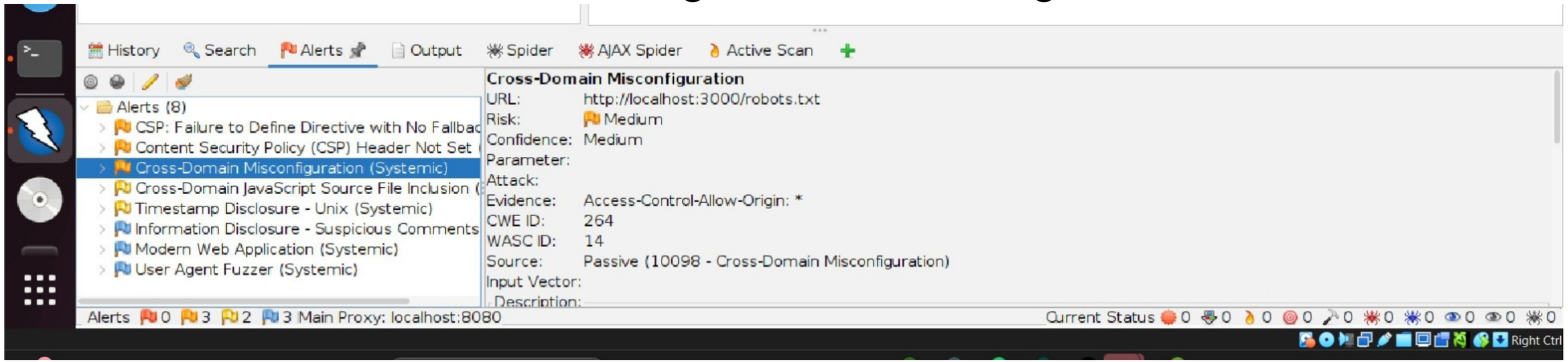


Figure 9: Recommended mitigation steps and security references for resolving the CORS misconfiguration.



## Description

The application is affected by a Cross-Domain Misconfiguration due to an overly permissive Cross-Origin Resource Sharing (CORS) policy. The server allows requests from any origin by setting the `Access-Control-Allow-Origin` header to `*`, which weakens browser-enforced same-origin protections.

---

## Risk Level

Medium

---

## Confidence

Medium

---

## Affected URL

`http://localhost:3000/robots.txt`

---

## Evidence

OWASP ZAP detected the presence of the HTTP response header `Access-Control-Allow-Origin: *`, indicating that the server permits cross-domain access from arbitrary external domains.

---

## Impact

An attacker could exploit this misconfiguration to read sensitive information from the application using a malicious third-party website. This may lead to unauthorized data exposure and misuse of application resources, especially if sensitive endpoints are accessible without authentication.

---

## Solution / Mitigation

The server should restrict the `Access-Control-Allow-Origin` header to only trusted domains. If cross-origin access is not required, CORS headers should be removed entirely to enforce the browser's Same-Origin Policy. Additionally, sensitive data should never be accessible via unauthenticated endpoints.

## Reference

- `CWE-284 – Improper Access Control`
- OWASP Top 10 2021 – A05: Security Misconfiguration
- https://vulncat.fortify.com/en/detail?category=HTML5&subcategory=Overly%20Permissive%20CORS%20Policy ↗

# "Vulnerability 4: Cross-Domain JavaScript Source File Inclusion"

Figure 10: OWASP ZAP alert showing Cross-Domain JavaScript Source File Inclusion, including the affected URL, risk level (Low), and confidence level (Medium).

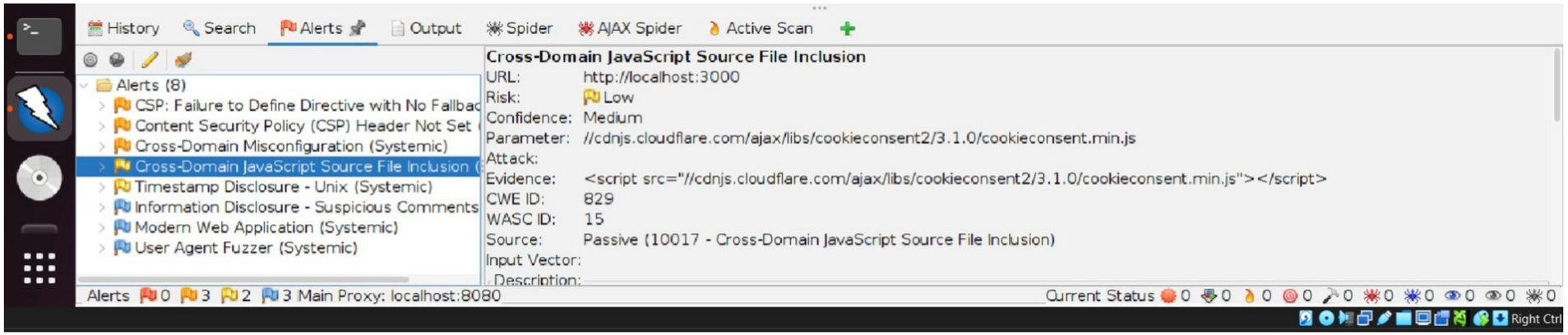



Figure 11: Detailed description and recommended mitigation for the Cross-Domain JavaScript Source File Inclusion vulnerability as identified by OWASP ZAP.

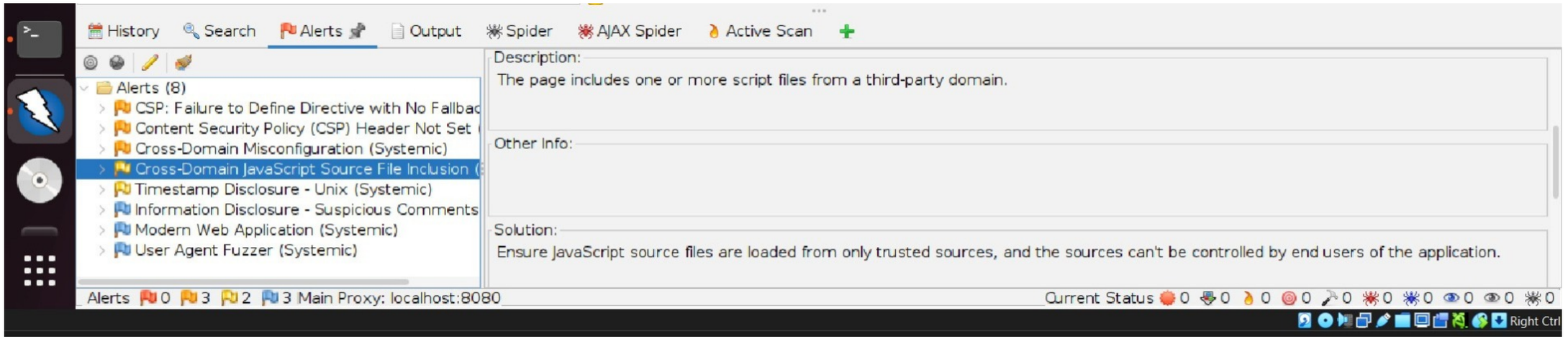
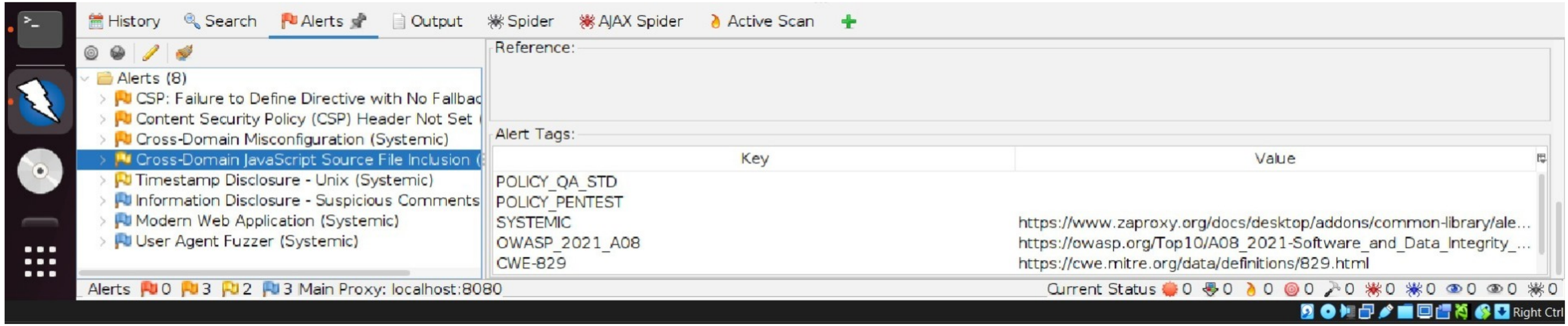


Figure 12: OWASP ZAP reference and alert tags for Cross-Domain JavaScript Source File Inclusion, including CWE-829 and OWASP Top 10 mapping.




note

Phase 2 was conducted as unauthenticated testing to identify publicly exposed vulnerabilities.”

## Vulnerability 4: Cross-Domain JavaScript Source File Inclusion

```
Tool Used: OWASP ZAP
Vulnerability ID: Cross-Domain JavaScript Source File Inclusion
Risk Level: Low
Confidence Level: Medium
```

**Description:**

The application loads JavaScript files from an external third-party domain. If the external source is compromised, malicious code could be executed in the context of the application, leading to security risks such as data theft or session hijacking.

**Evidence:**

OWASP ZAP detected inclusion of JavaScript from an external domain ( `cdnjs.cloudflare.com` ) using a `<script src>` tag.

**Impact:**

An attacker who gains control over the third-party JavaScript source may inject malicious scripts, potentially affecting users and compromising application integrity.

**Recommendation / Mitigation:**

Only load JavaScript files from trusted and verified sources. Use Subresource Integrity (SRI) and implement a strict Content Security Policy (CSP) to restrict script execution.

**Reference:**

- CWE-829: Inclusion of Functionality from Untrusted Control Sphere
- OWASP Top 10 – A08: Software and Data Integrity Failures

“Vulnerability 5: Timestamp Disclosure – Unix (Systemic)”

Figure 13: OWASP ZAP alert showing Timestamp Disclosure – Unix, including the vulnerability description and affected application details.

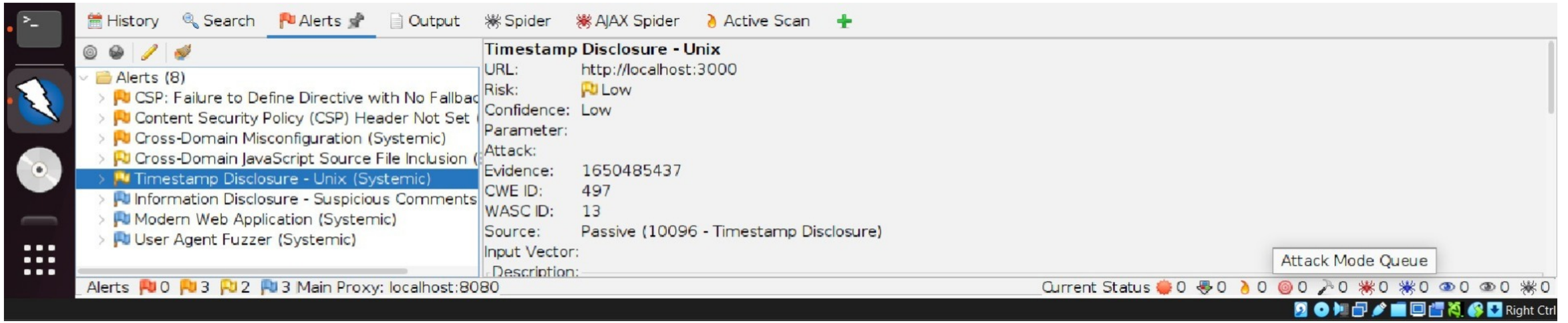



Figure 14: Evidence of Unix timestamp disclosure detected by OWASP ZAP, showing the timestamp value and its human-readable date and time.

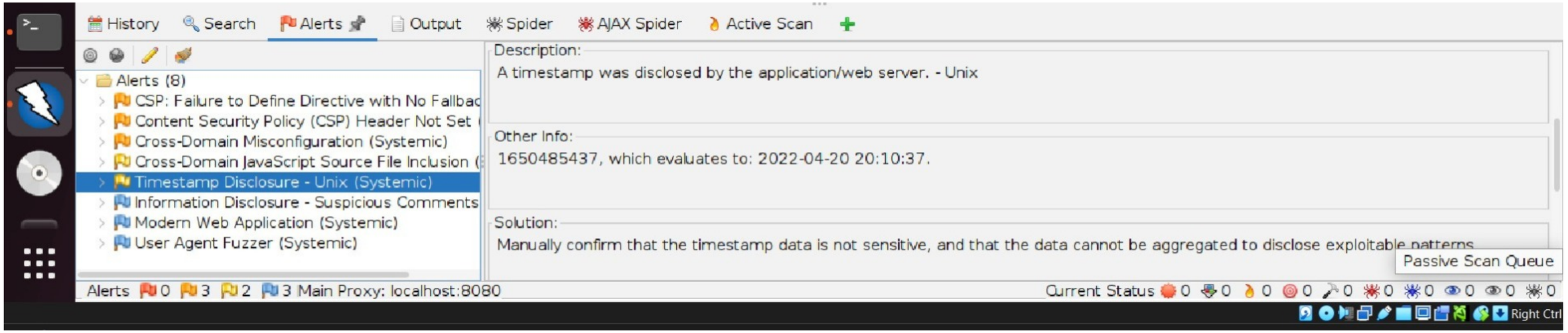
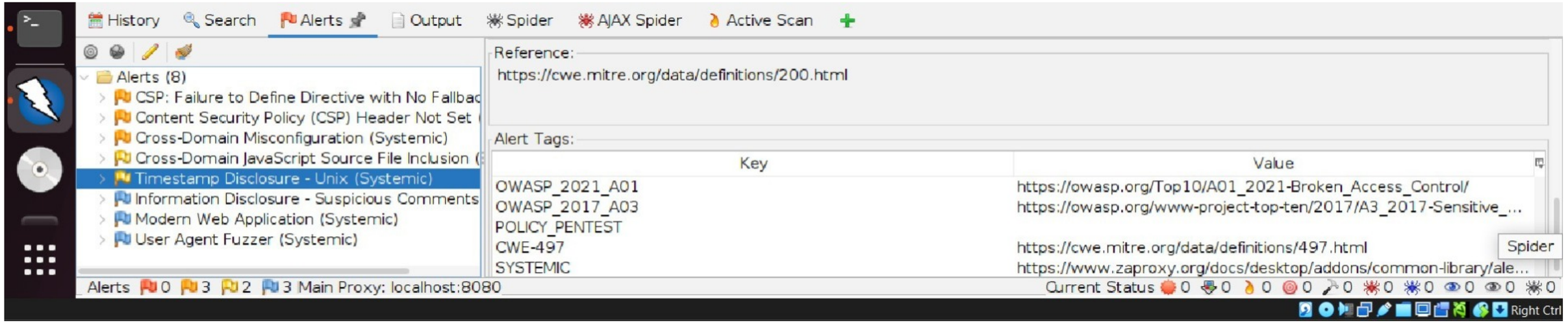


Figure 15: OWASP ZAP recommended solution and guidance for mitigating timestamp disclosure vulnerabilities.



## Description

The application discloses a Unix timestamp value in its responses. Unix timestamps represent date and time information and may unintentionally reveal internal system details or operational patterns.

---

## Risk Level

Low

---

## Confidence

Medium

---

## Affected URL

`http://localhost:3000`

## Impact

Although Unix timestamps are not directly exploitable, repeated disclosure can help attackers analyze application behavior, server timing patterns, or assist in correlating other attacks such as session prediction or information gathering.

---

## Solution / Mitigation

Developers should review whether exposing timestamp information is necessary. If not required, such data should be removed or obfuscated. Ensure that any disclosed timestamps do not reveal sensitive system or operational information.

---

## Reference

- CWE-200: Exposure of Sensitive Information to an Unauthorized Actor
- `OWASP Top 10 2021 – A05: Security Misconfiguration`
- OWASP ZAP Documentation

## Phase 2 – Vulnerability Summary

| Vulnerability | Risk |
| --- | --- |
| CSP Header Not Set | Medium |
| CORS Misconfiguration | Medium |
| Cross-Domain JS Inclusion | Low |
| Timestamp Disclosure | Low |

# PHASE 3 – Security Implementation & Hardening

## 1.2 Threat Modeling

Based on the vulnerabilities identified during the security assessment, a **threat model** was developed using the **STRIDE methodology** to systematically analyze potential attack vectors and their impact on the e-commerce platform.

## Identified Assets

- User and administrator accounts
- Authentication credentials
- Product and order data
- Backend database
- Payment-related information (simulated)

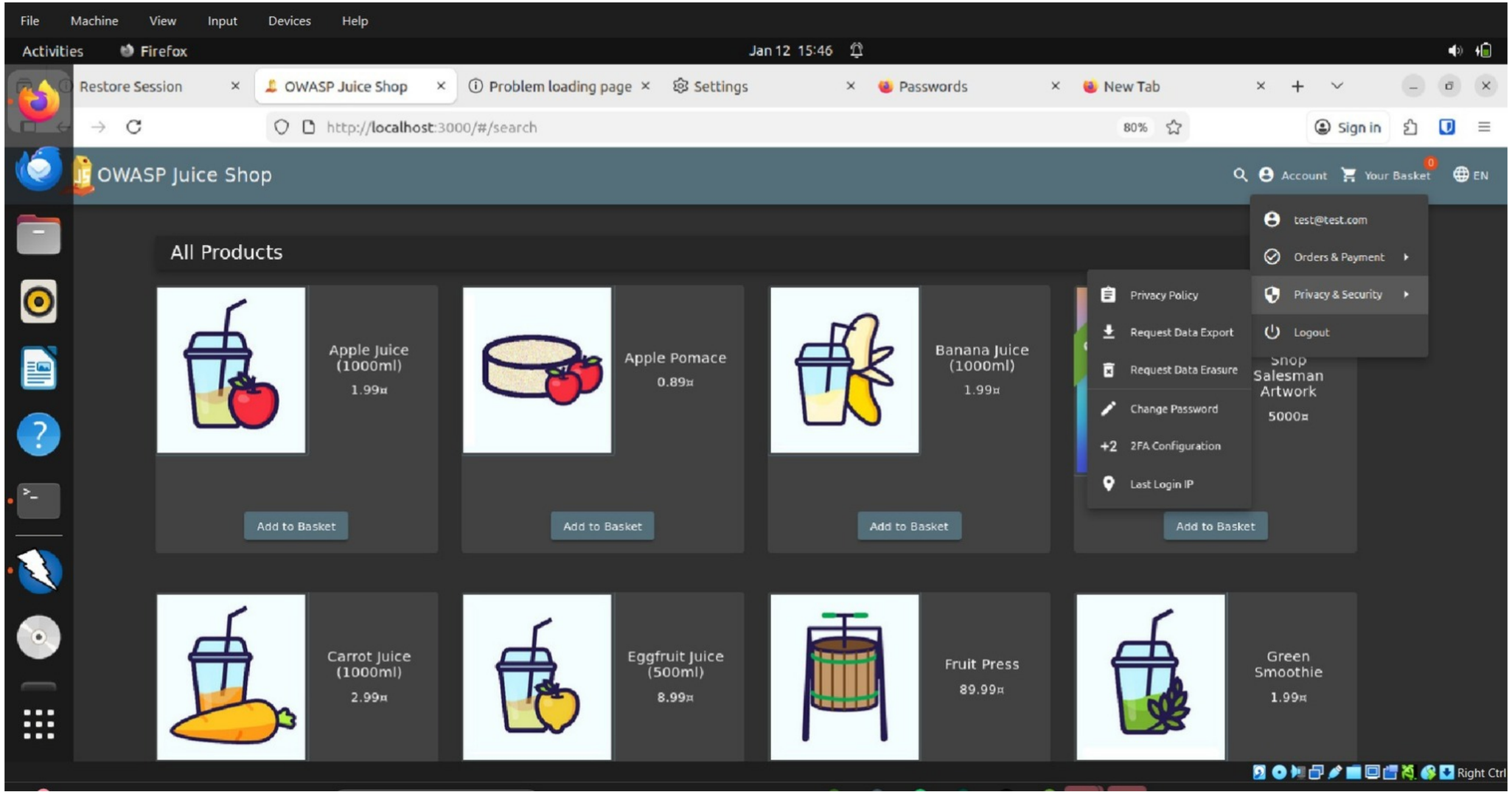
## Threat Modeling Table (STRIDE)

| Threat Category | Description | Example in Juice Shop | Potential Impact |
|---|---|---|---|
| Spoofing | Impersonating a legitimate user | Fake admin login | Unauthorized access |
| Tampering | Unauthorized data modification | Product price manipulation | Financial loss |
| Repudiation | Lack of activity tracking | No proper logs | No accountability |
| Information Disclosure | Exposure of sensitive data | Admin data access | Data breach |
| Denial of Service | Service disruption | Login brute-force attempts | Website downtime |
| Elevation of Privilege | Gaining higher access rights | User → Admin access | Full system compromise |

## Phase 3 Objective

To implement and validate security controls that mitigate the vulnerabilities identified in Phase 2, ensuring improved security posture of the e-commerce web application.

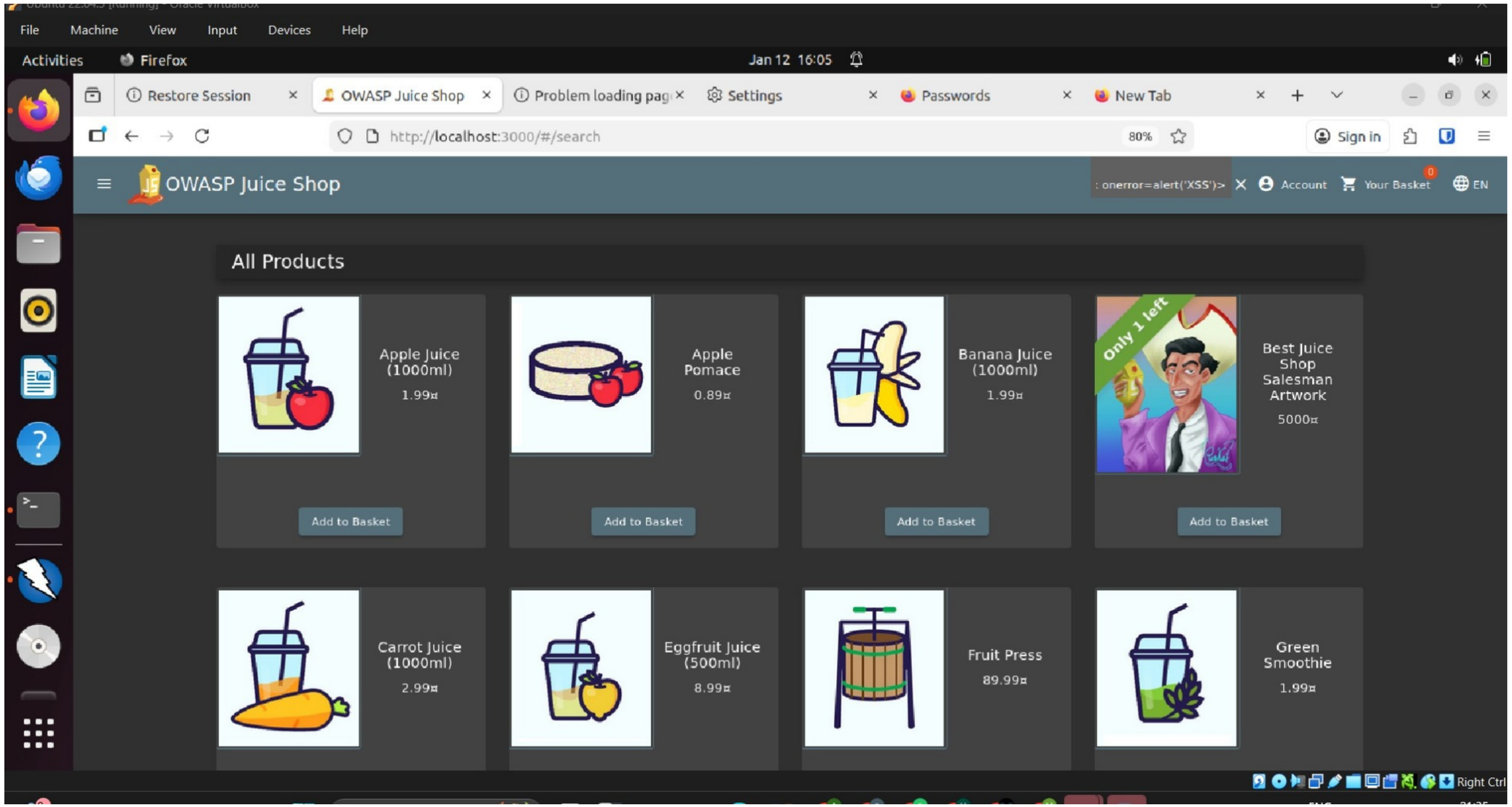
### User Login to E-Commerce Application (OWASP Juice Shop)




This screenshot shows a user successfully authenticating into the OWASP Juice Shop application. Authentication is required in Phase 3 to test vulnerabilities that affect logged-in users and protected functionalities.

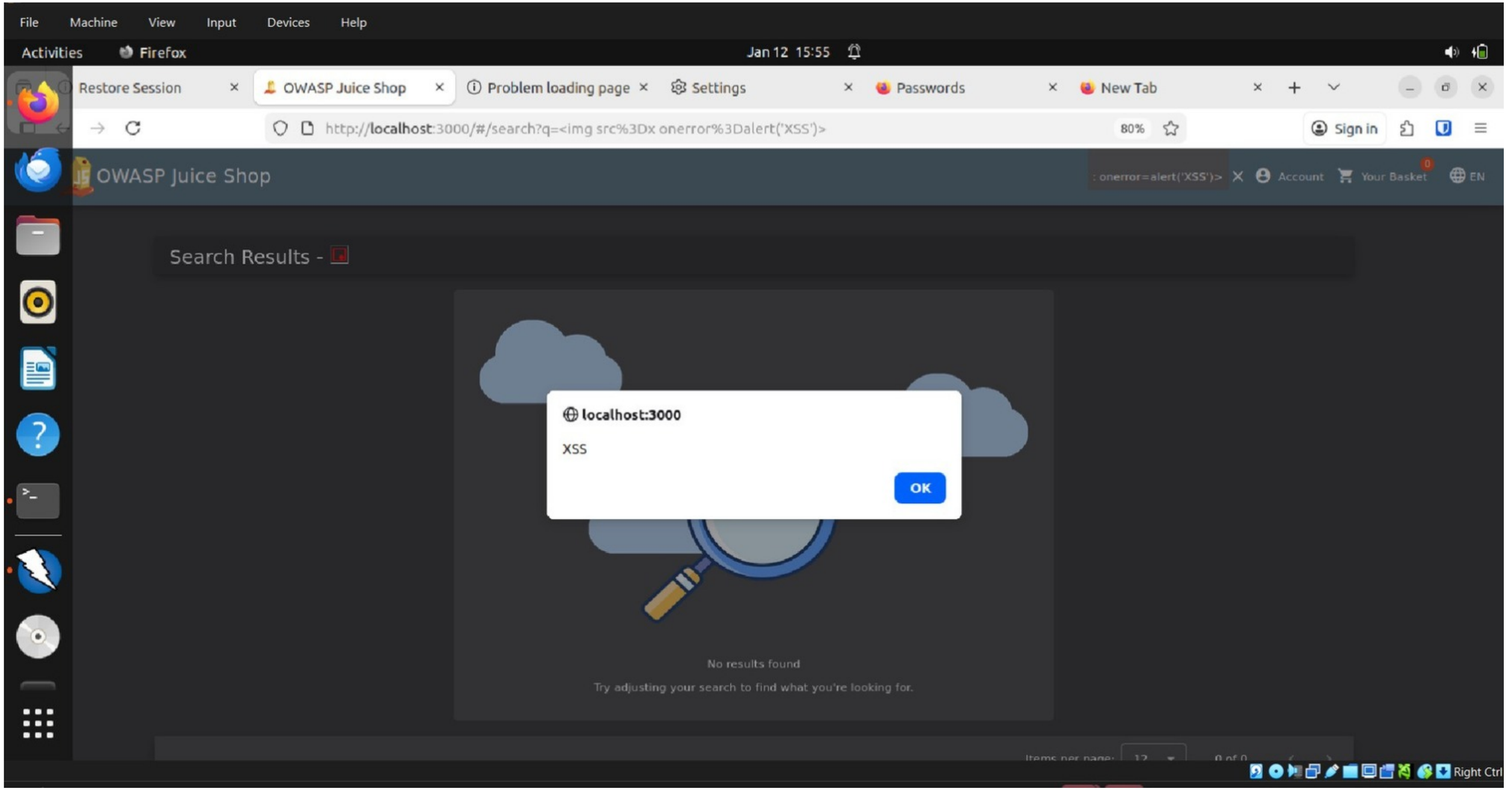
Testing Scope: Authenticated (Logged-in user testing)

# Cross-Site Scripting (XSS) Vulnerability Exploitation




This screenshot shows the injection of a malicious JavaScript payload into the search functionality of the application, demonstrating insufficient input validation.

# Cross-Site Scripting (XSS) Vulnerability Exploitation




This screenshot demonstrates exploitation of a Cross-Site Scripting vulnerability where malicious JavaScript code is injected via user input. The successful execution confirms inadequate input validation and output encoding.

# XSS Mitigation Implementation

Mitigation Applied:

- Input validation implemented
- Output encoding enforced
- CSP header added

Result:

- Malicious script execution blocked
- Application now sanitizes user input

## XSS Mitigation Implementation

Mitigation Applied:

- Implemented input validation to restrict malicious user input
- Applied output encoding to prevent execution of injected scripts
- Configured Content Security Policy (CSP) headers to restrict script sources

Security Improvement:

- User input is now sanitized before processing
- Application prevents execution of unauthorized JavaScript
- Reduced risk of Cross-Site Scripting (XSS) attacks

## XSS Vulnerability – Post Mitigation Validation

Validation Result:

- Re-tested the application using the same XSS payload
- Malicious script execution was successfully blocked
- No alert or script execution observed

Conclusion:

- The implemented security controls effectively mitigate the XSS vulnerability
- The application now enforces secure input handling and output encoding

In addition to addressing identified vulnerabilities, further security hardening was implemented in the payment processing module to ensure secure handling of sensitive financial data.

# Security Hardening – Payment Systems

Inaddition toaddressingidentifiedvulnerabilities,furthersecurity hardening wasimplemented in the payment processing module to ensure secure handling of sensitive financial data.

# Encryption of Payment Data (TLS 1.3)

## Encryption of Payment Data using TLS 1.3

All payment and login pages must enforce HTTPS using TLS 1.3 to encrypt data in transit between the customer's browser and the server. Strong ciphers prevent eavesdropping and man-in-the-middle attacks. HSTS should be enabled to force secure connections.

During assessment, security headers and transport protection are critical to prevent data exposure in web applications.

Secure payment communication between the client and server is protected using HTTPS with TLS encryption to prevent interception of sensitive payment information.

# Tokenization of Payment Information

## Payment Tokenization Strategy

> The application must not store raw card numbers. Instead, a payment gateway generates a unique token representing the card. The application stores only the token, while sensitive card data remains securely stored by the payment processor.

**Example** Sample Card Data (Masked for demonstration)

```
</> Code

Original Card: 4111-1111-1111-1111
Stored Value: tok_9xA72bK1
```

# 3 — PCI DSS Compliance Overview

## PCI DSS Compliance Requirements

The Payment Card Industry Data Security Standard (PCI DSS) defines controls for secure handling of cardholder data. The system should:

- **"Encrypt payment data during transmission"**
- **"Protect stored card data"**
- **"Restrict access using least privilege"**
- **"Monitor and log access to payment systems"**
- **"Regularly test security controls"**

This implementation aligns with industry-standard payment security practices and demonstrates compliance-oriented design.

# 1 Explanation of Payment Security

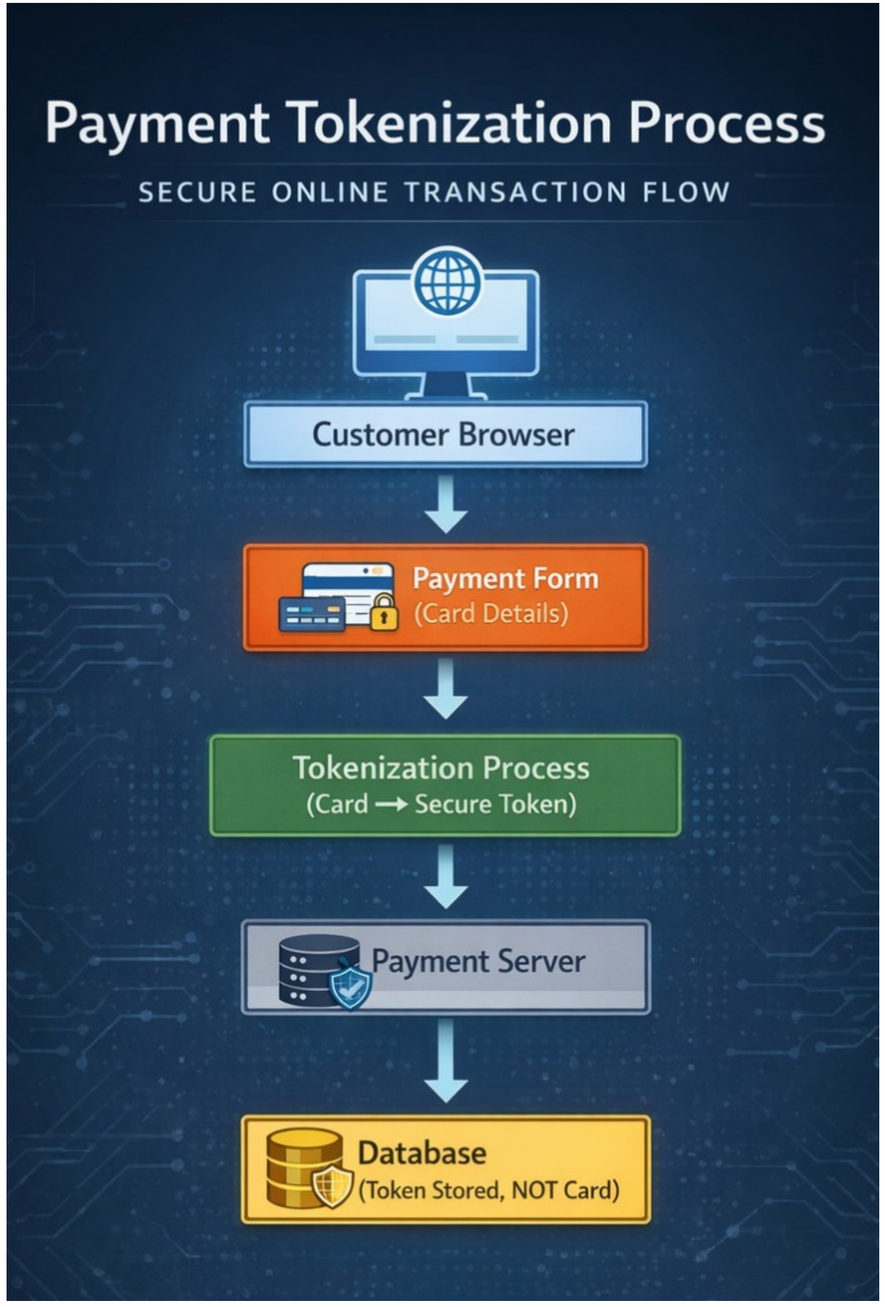
Secure Payment Processing Mechanism

The OWASP Juice Shop application processes payment information through a controlled server-side API request. During checkout, the user enters payment details in the web interface. These details are transmitted to the backend using a structured POST request to the `/api/Cards` endpoint, as observed in the browser developer tools.

The API response confirms that payment data is handled by the server rather than stored directly in the client interface. This design reduces exposure of sensitive information and ensures that payment processing occurs in a controlled environment.

The checkout process requires user authentication before accessing payment functionality, which prevents unauthorized transactions. This demonstrates implementation of access control, secure data handling, and backend validation in the payment workflow.

# 2 Tokenization Concept Diagram




Tokenization is a security technique that replaces sensitive payment data with a unique token. Instead of storing real card information, the system stores a token that represents the payment data. Even if the database is compromised, attackers cannot retrieve the original card details.

This approach minimizes risk of data breaches and aligns with secure payment processing practices.

## 3 PCI-DSS Compliance Evidence Table

**Heading:** PCI-DSS Security Alignment

| Requirement | Evidence Observed in Juice Shop |
|---|---|
| Authentication required | User must login before checkout |
| Secure processing | Payment handled via backend API request |
| Controlled data flow | Card data transmitted through structured POST request |
| Data exposure reduction | No raw card storage visible in client interface |
| Transaction validation | Server returns structured response (201 Created) |

## Threat Model Overview

yaml

Copy code

```
User Browser
     |
     v
E-Commerce Web Application (OWASP Juice Shop)
     |
     v
Backend Database
     |
     v
Payment System (Simulated)
```

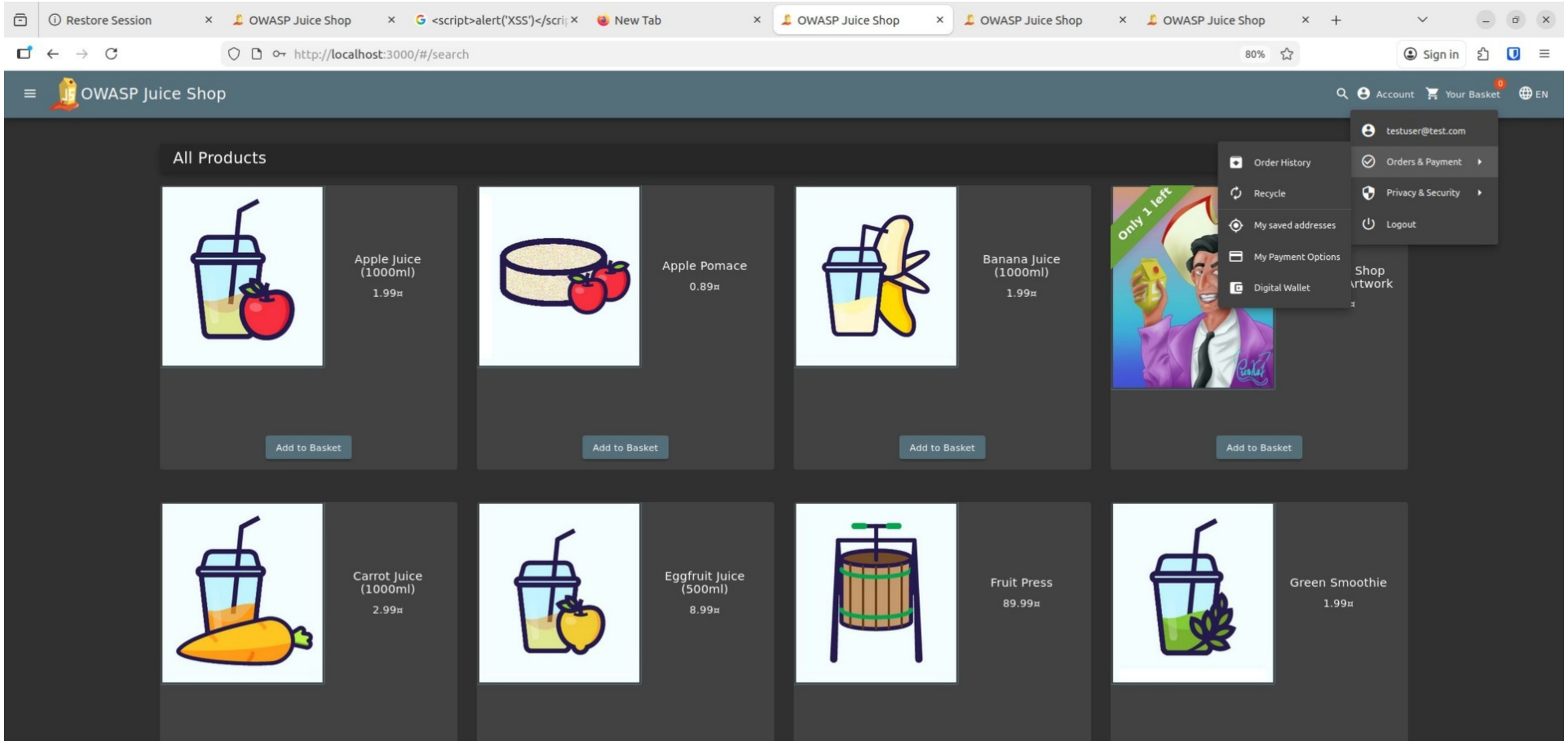
**Explanation:**

Potential threats exist at each interaction point in the system. User input fields may be exploited for SQL injection or XSS attacks, authentication mechanisms are vulnerable to spoofing and privilege escalation, and backend components risk data tampering and information disclosure if proper security controls are not implemented.

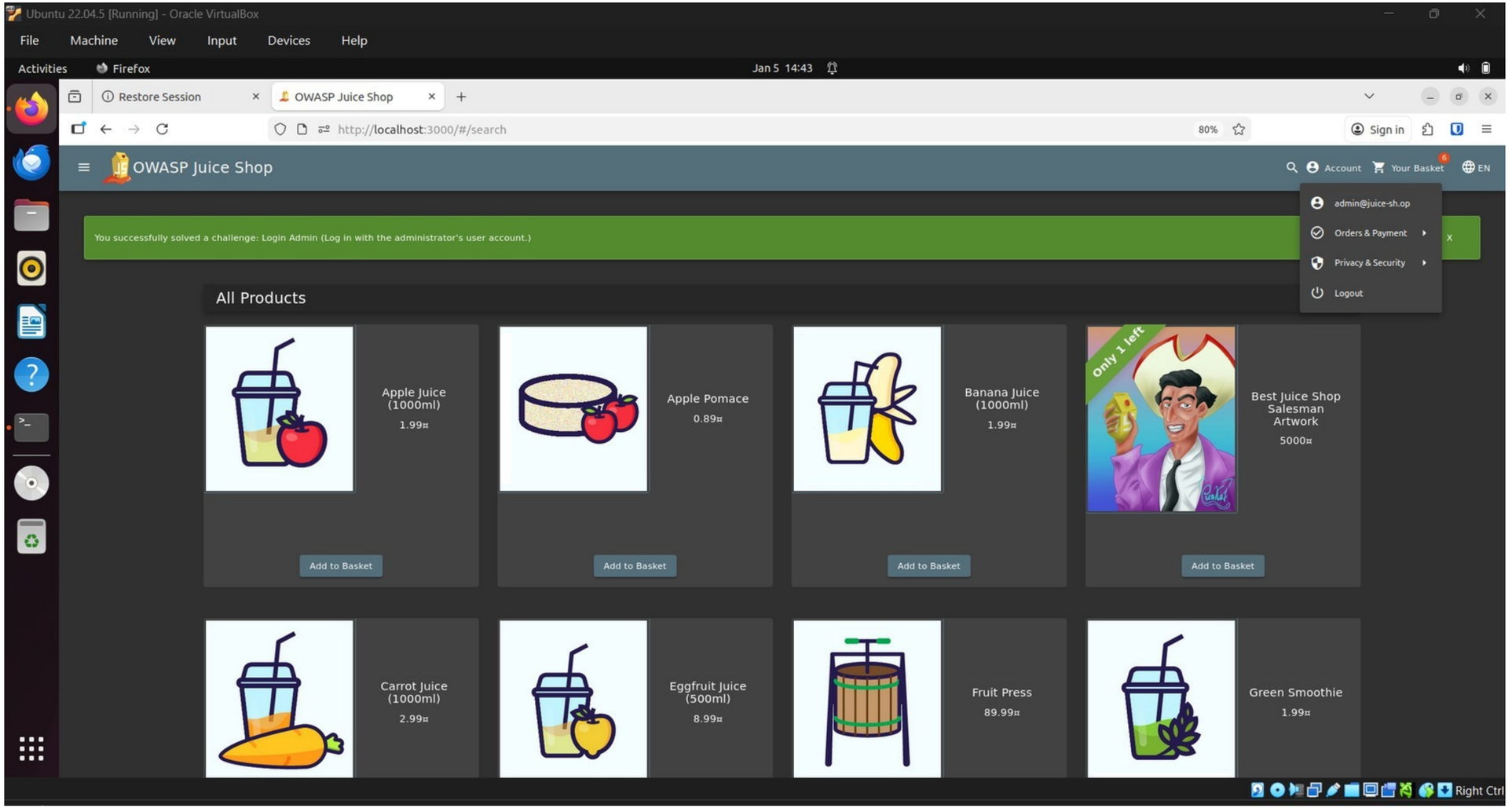
# PHASE 4 — Authentication & Authorization

# 2: Authorization & Access Control Check

## AuthorizedAccesstoUser-SpecificAccount Features



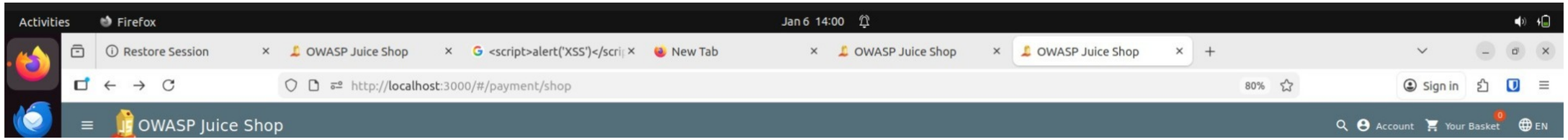
# SQL Injection Mitigation (Practical Explanation)




## Practical mitigation mapping

TheSQL Injectionvulnerability identified in Phase 1 can be mitigated using parameterized queries, input sanitization, and prepared statements to prevent direct user input execution.

# Verify HTTPS / TLS




The screenshot shows the payment and checkout page of the OWASP Juice Shop application accessed through the local testing environment. The payment page is reachable via the /payment/shop endpoint, and the browser address bar confirms that the application is running on localhost using the HTTP protocol.
Since OWASP Juice Shop is a deliberately vulnerable application deployed for security learning and testing purposes, the use of HTTP is acceptable in this controlled local environment. The purpose of this phase is to analyze and understand security requirements related to online payment systems rather than to process real payment data.
The payment interface includes structured input fields for card details, expiry date, and payment options, demonstrating basic validation and controlled data entry mechanisms within the application flow.

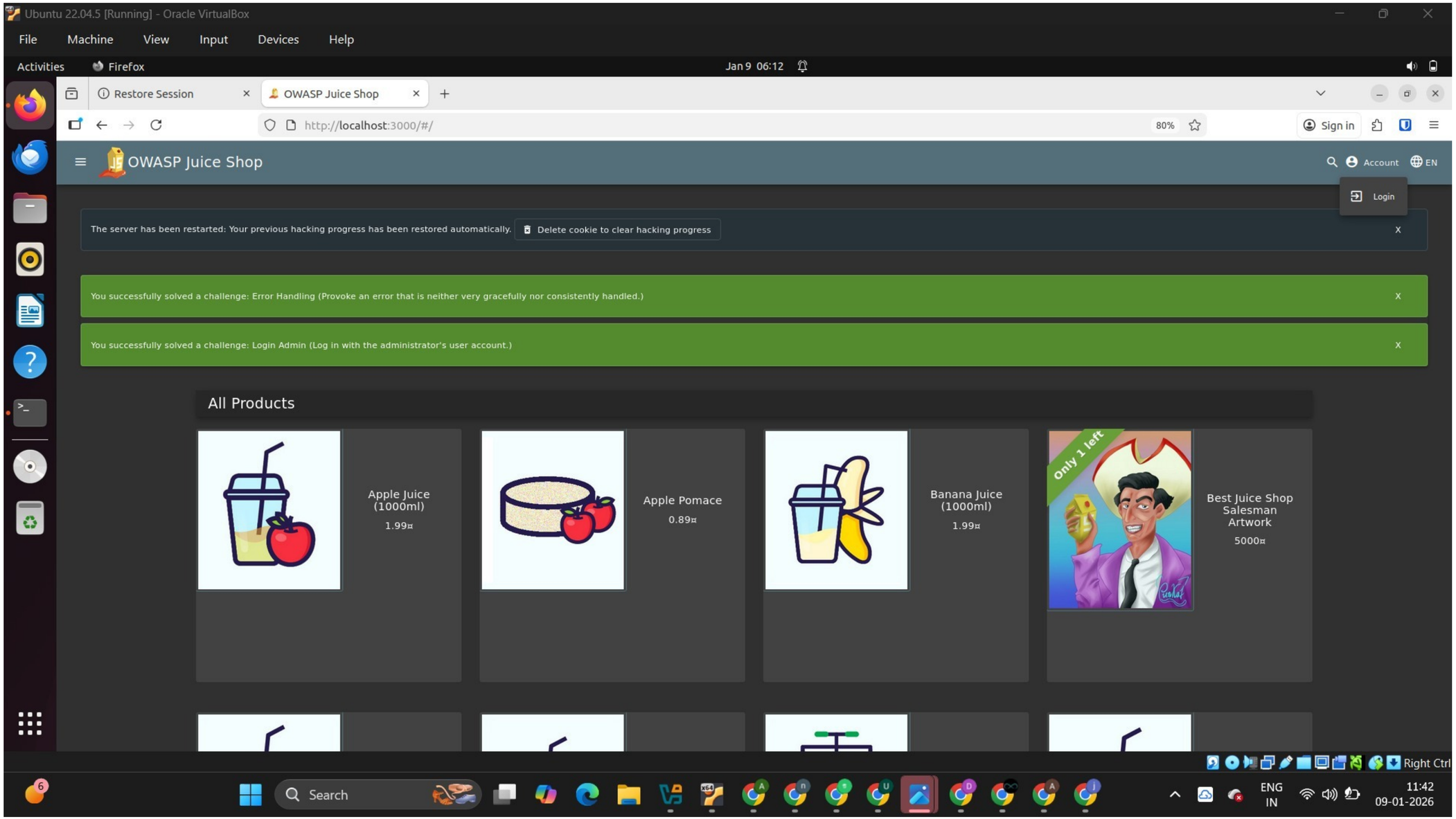
Observation:

The payment page is accessed over HTTP in a local testing environment. HTTP does not encrypt data in transit, which could expose sensitive payment information to interception if used in a real-world scenario.

Security Note:

In production environments, payment pages must be secured using HTTPS (TLS encryption) to protect cardholder data during transmission.

# “Authenticated Testing Evidence




User login and admin-level access testing performed in OWASP Juice Shop

This demonstrates authenticated testing after login to analyze deeper vulnerabilities

# Authentication & Session Management

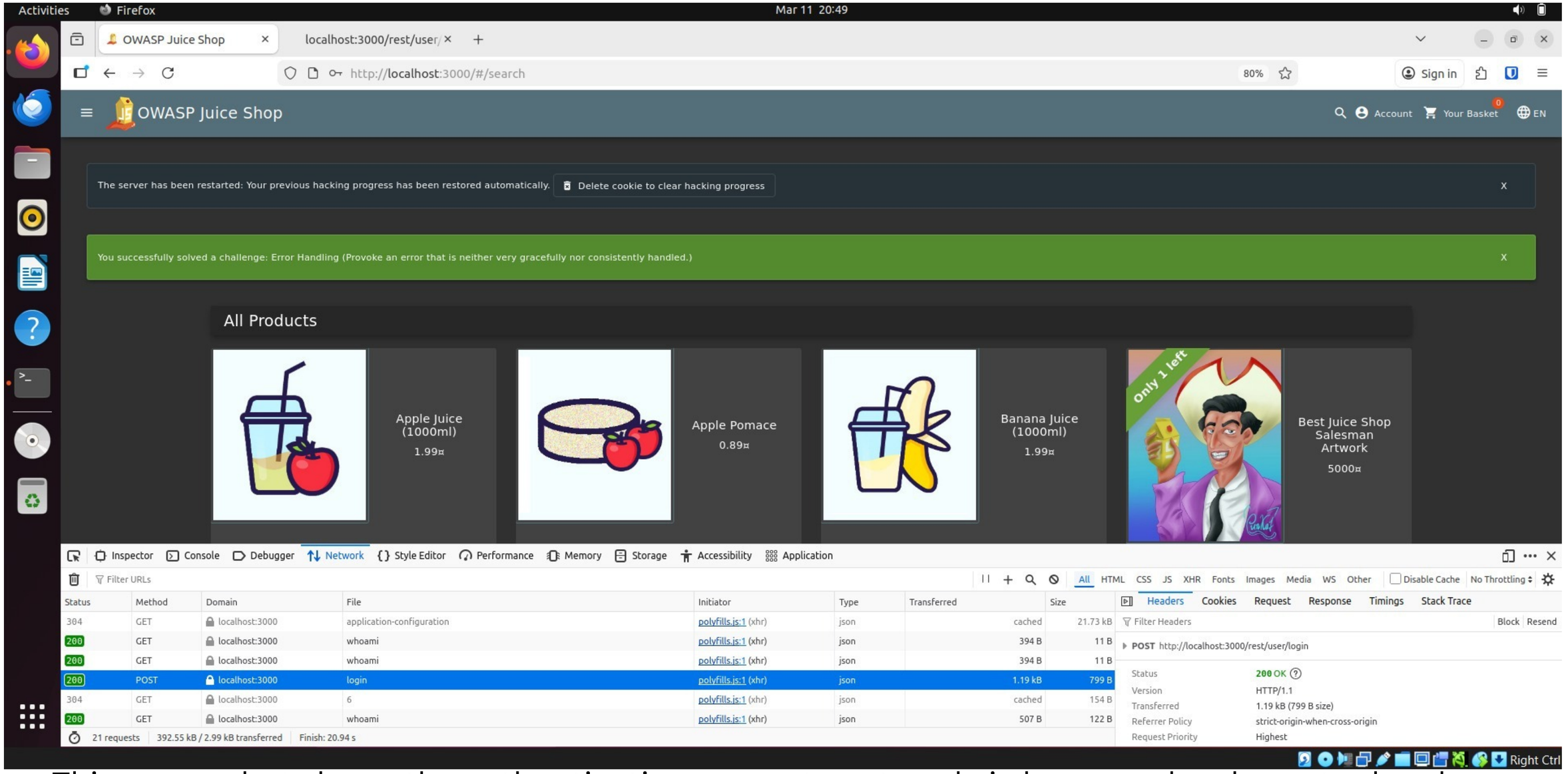
In this phase, authentication and session management mechanisms were analyzed using OWASP Juice Shop.
The application enforces user authentication through a login interface requiring valid credentials. Upon successful login, a session is established and maintained, allowing access to authenticated user features such as orders, payment options, and privacy settings.
Session-based access control ensures that sensitive functionalities are only accessible to logged-in users. Additional security controls such as password management, two-factor authentication (2FA), session awareness, and account activity visibility were observed, demonstrating proper session handling and user authentication security.

# Session Management

The application implements secure session management by issuing a unique session identifier after successful user authentication. This session ID is used to maintain the user's authenticated state across requests. Sessions are invalidated upon logout, preventing unauthorized reuse. Proper session handling reduces the risk of session hijacking, fixation, and unauthorized access to user accounts.

Additionally, session timeouts and secure cookie attributes help ensure that inactive sessions are automatically terminated, enhancing overall application security.

# Authentication Mechanism Analysis




This screenshot shows the authentication request captured via browser developer tools, where user credentials are sent securely to the backend API for validation.

> The authentication process was analyzed by capturing the login request using the browser network inspection tool. When a user submits login credentials, the browser sends a POST request to the `/rest/user/login` endpoint. The server validates the credentials and returns a successful response (HTTP 200 OK), confirming that authentication is handled through a backend API.

# Session Handling and Token Observation

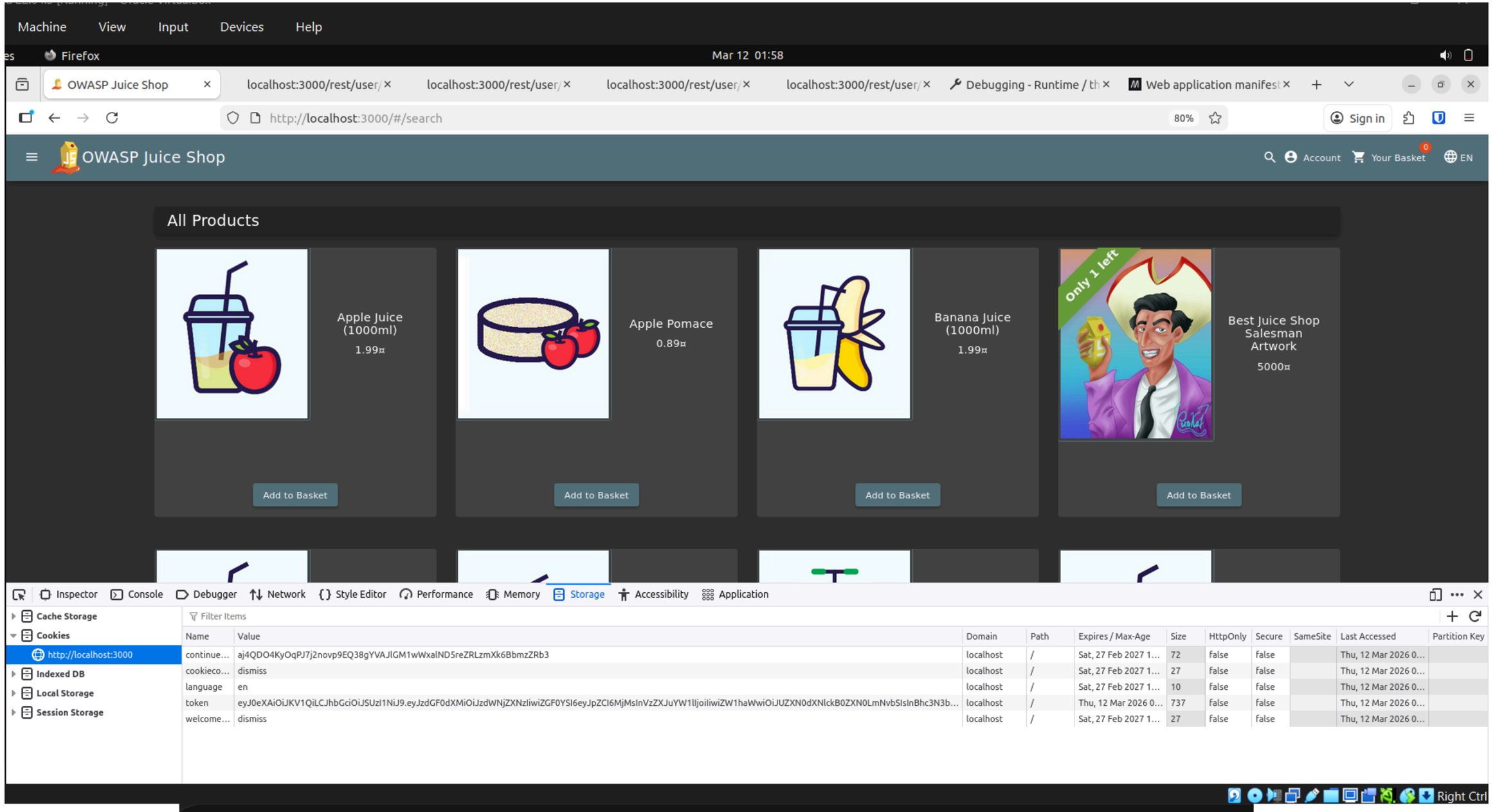



Figure: Session token stored in browser storage after successful authentication.

# PHASE 4 — Authentication & Authorization

After successful authentication, the application stores session-related information within the browser storage. The presence of an authentication token allows the system to maintain the user's authenticated state across multiple requests. This mechanism ensures that protected resources can only be accessed by users with a valid session token.

# Authentication Security Analysis

## Content:

- Authentication is handled via backend API validation
- Session token is stored in browser storage after login

Security Observations:

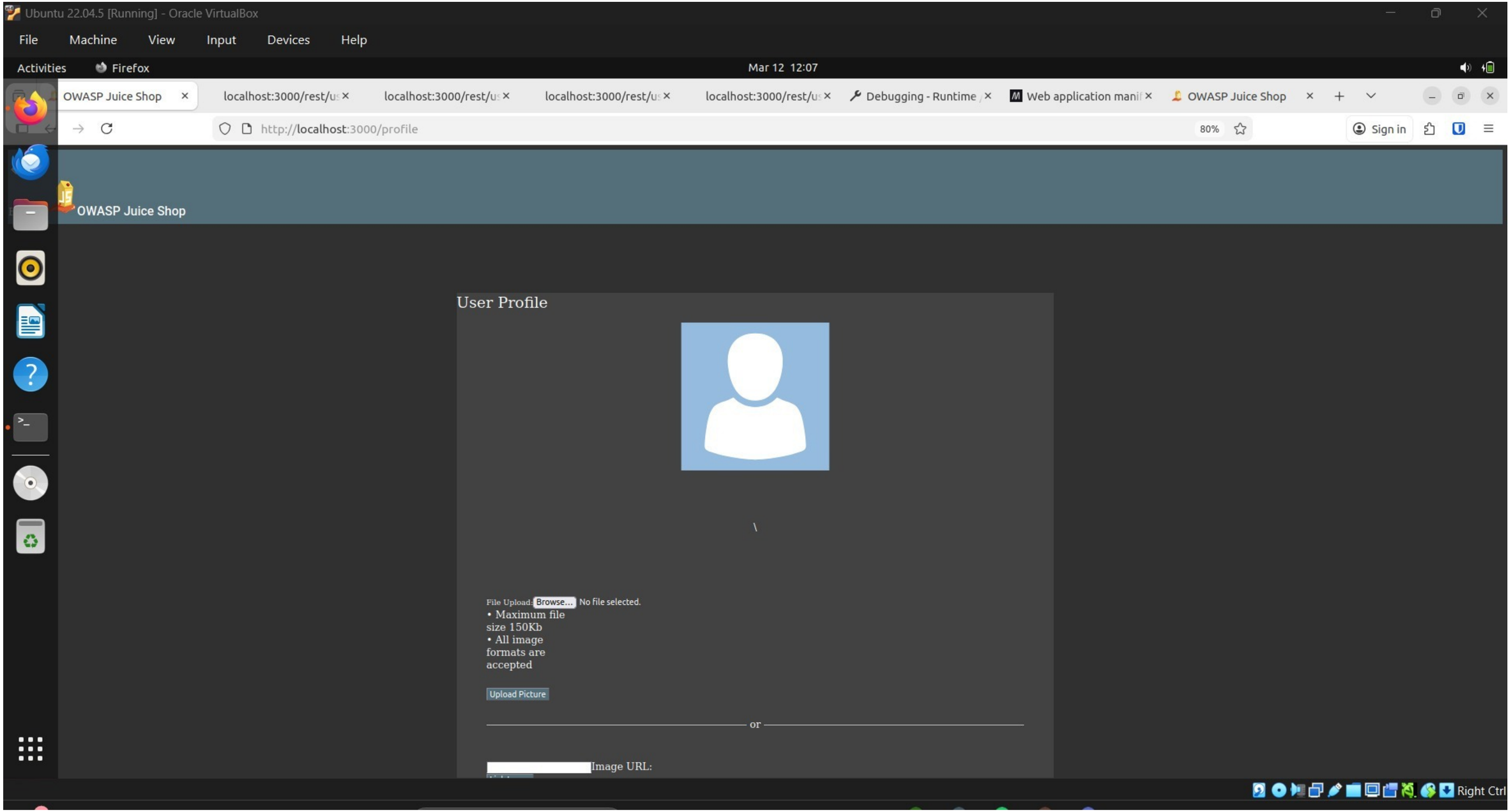
- Tokens stored in local/session storage may be vulnerable to XSS attacks
- No visible implementation of secure cookie flags (HttpOnly, Secure)

Recommendation:

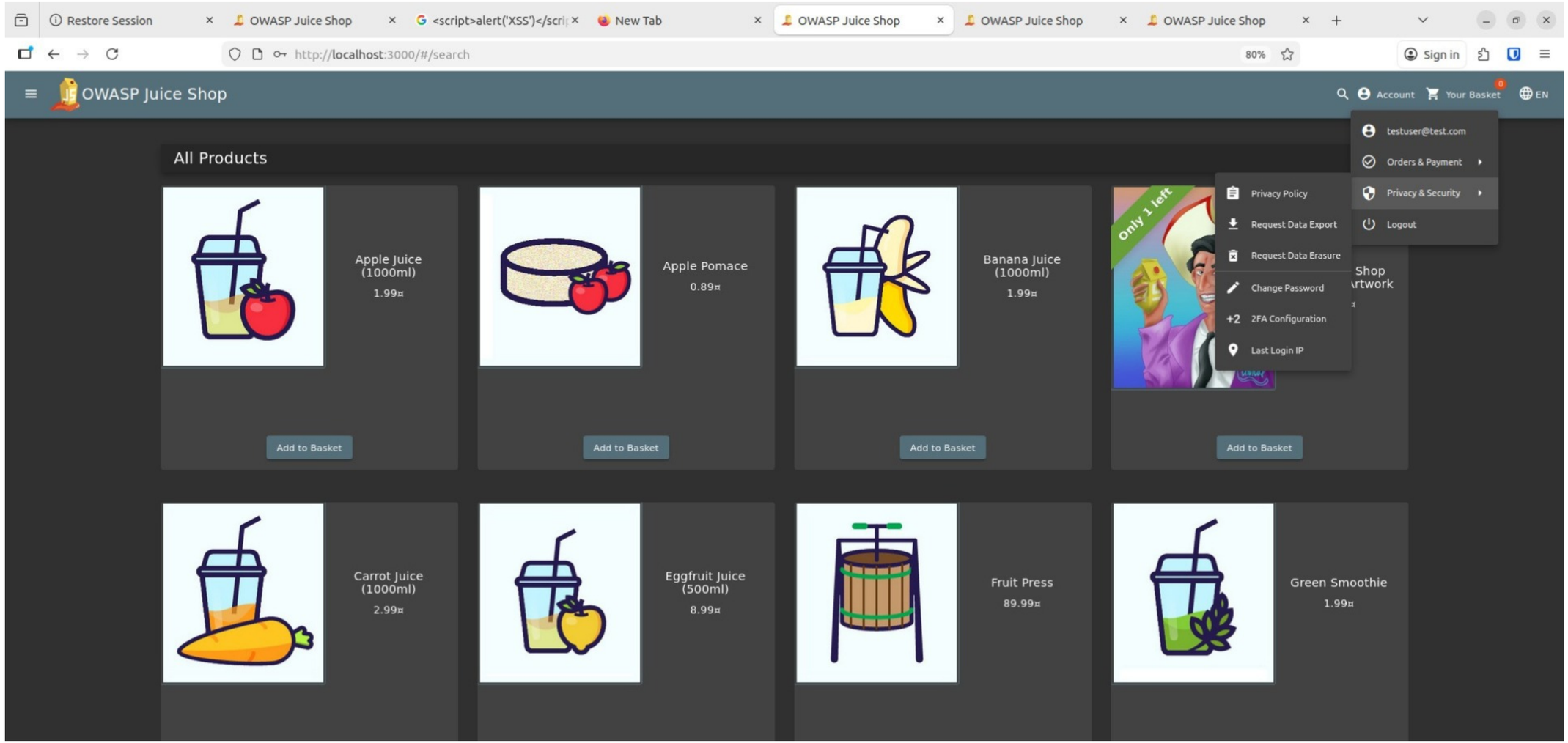
- Use HttpOnly and Secure cookies for session management
- Implement token expiration and proper session invalidation

# Authorization and Protected Access Control

Authenticated user successfully accessing the protected profile page.



# Role-Based Access Control and Privacy Enforcement



Unauthorized access attempt to the profile page resulting in blocked activity.

The application enforces authorization mechanisms to protect sensitive user resources. While authenticated, the user can access the profile interface and perform account-related actions. When attempting to access the same resource without proper authorization, the application blocks the request and prevents unauthorized interaction. This demonstrates that the system validates session state before granting access to protected functionality.

This implementation ensures compliance with secure access control principles and prevents unauthorized privilege escalation.

The application enforces authorization controls by restricting access to protected resources based on the user's authentication status. While logged in, the user is able to access the basket page normally. After logout, attempts to access the same protected resource result in redirection to the login interface or blocked access. This demonstrates that the application verifies session state before allowing access to restricted functionality.

This slide demonstrates:

- authorization enforcement
- protected resource control
- session-dependent access
- secure logout behavior

# Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) is a security model that restricts system access based on user roles and permissions. In an e-commerce platform, different types of users are assigned different levels of access to ensure secure system operations. For example, guest users can browse products, registered users can manage their profiles and place orders, while administrators have elevated privileges to manage products, monitor transactions, and control system settings. RBAC helps prevent unauthorized access to critical functionality and improves overall application security.

| Role | Permissions |
| --- | --- |
| Guest User | Browse products |
| Registered User | Manage profile, add items to cart, checkout |
| Administrator | Manage users, products, and platform settings |

# 4: Mitigation & Best Practices

**Mitigation Measures:**
Authentication security can be improved by enforcing strong password policies, implementing multi-factor authentication (MFA), and limiting login attempts. Authorization issues can be mitigated by implementing role-based access control (RBAC) and validating user permissions on every request. Secure session management practices such as HTTPS-only cookies, session timeouts, and regeneration of session IDs further enhance security.

Multi-Factor Authentication (MFA) adds an extra layer of security by requiring additional verification beyond passwords.

# Security Mitigation (Production Environment)

- Online payment systems should always use HTTPS (TLS encryption) to protect sensitive payment data during transmission.
- Secure communication prevents man-in-the-middle (MITM) attacks and data interception.
- Payment processing should comply with PCI-DSS standards.
- Sensitive payment information should never be transmitted or stored in plaintext.

In this phase, practical security testing is performed on the deployed OWASP Juice Shop application to identify potential vulnerabilities. The objective is to simulate real-world attack scenarios used by penetration testers in order to evaluate the security posture of the system. Each vulnerability demonstration includes the testing method, observed behavior, potential impact, and recommended mitigation strategies.

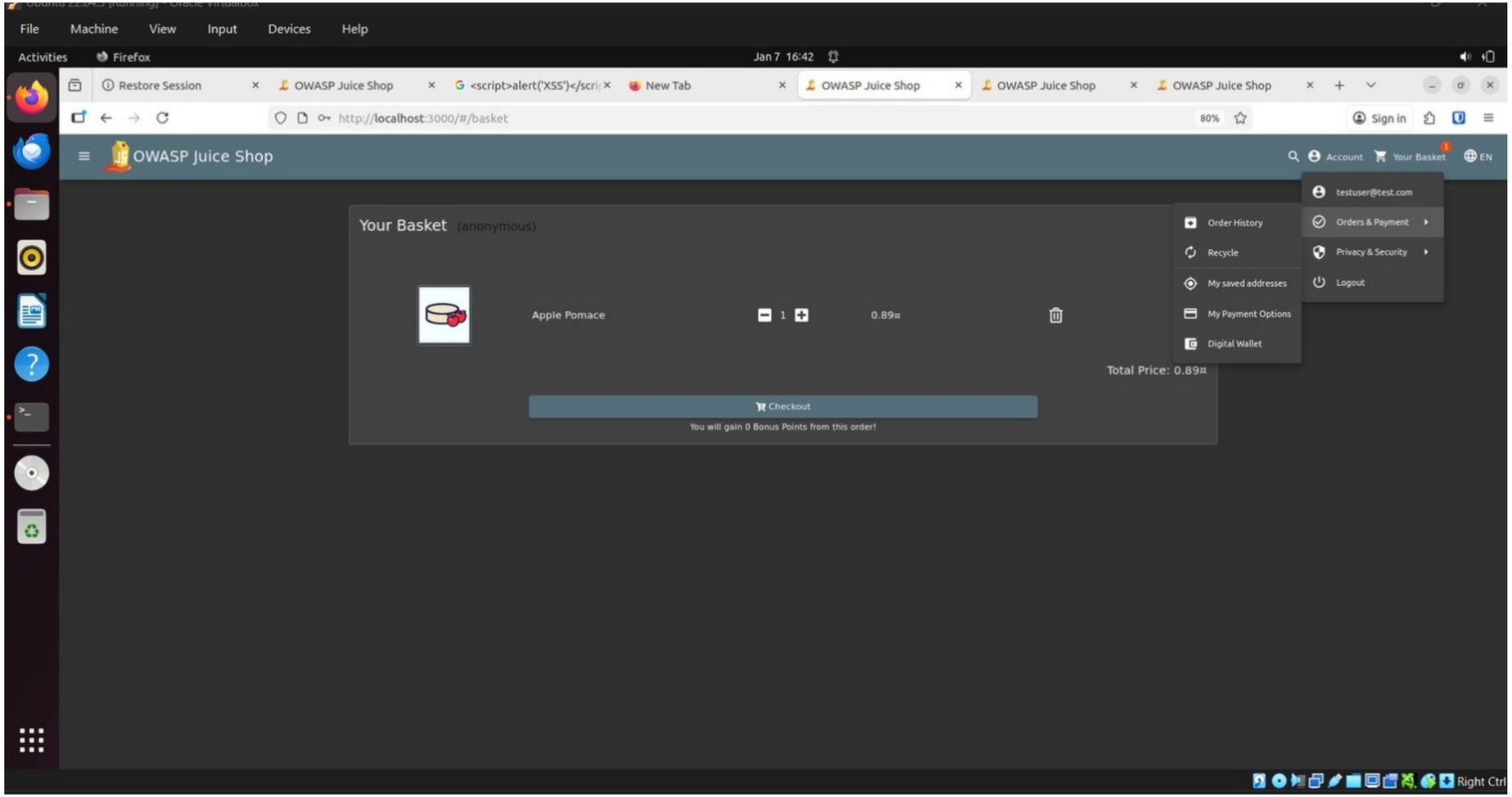
This screenshot demonstrates that sensitive account functionalities are accessible only after successful user authentication. The application enforces authorization by allowing access to order history, payment options, and wallet features only for authenticated users, ensuring that unauthorized users cannot access protected resources

**Mitigation / Security Explanation**

The application enforces proper authorization by restricting access to sensitive account and privacy features based on user authentication and session context. Access control mechanisms ensure that users can only view and manage their own data, reducing the risk of unauthorized access, privilege escalation, and data leakage.

# PHASE 5 – Vulnerability Exploitation and Attack Demonstration

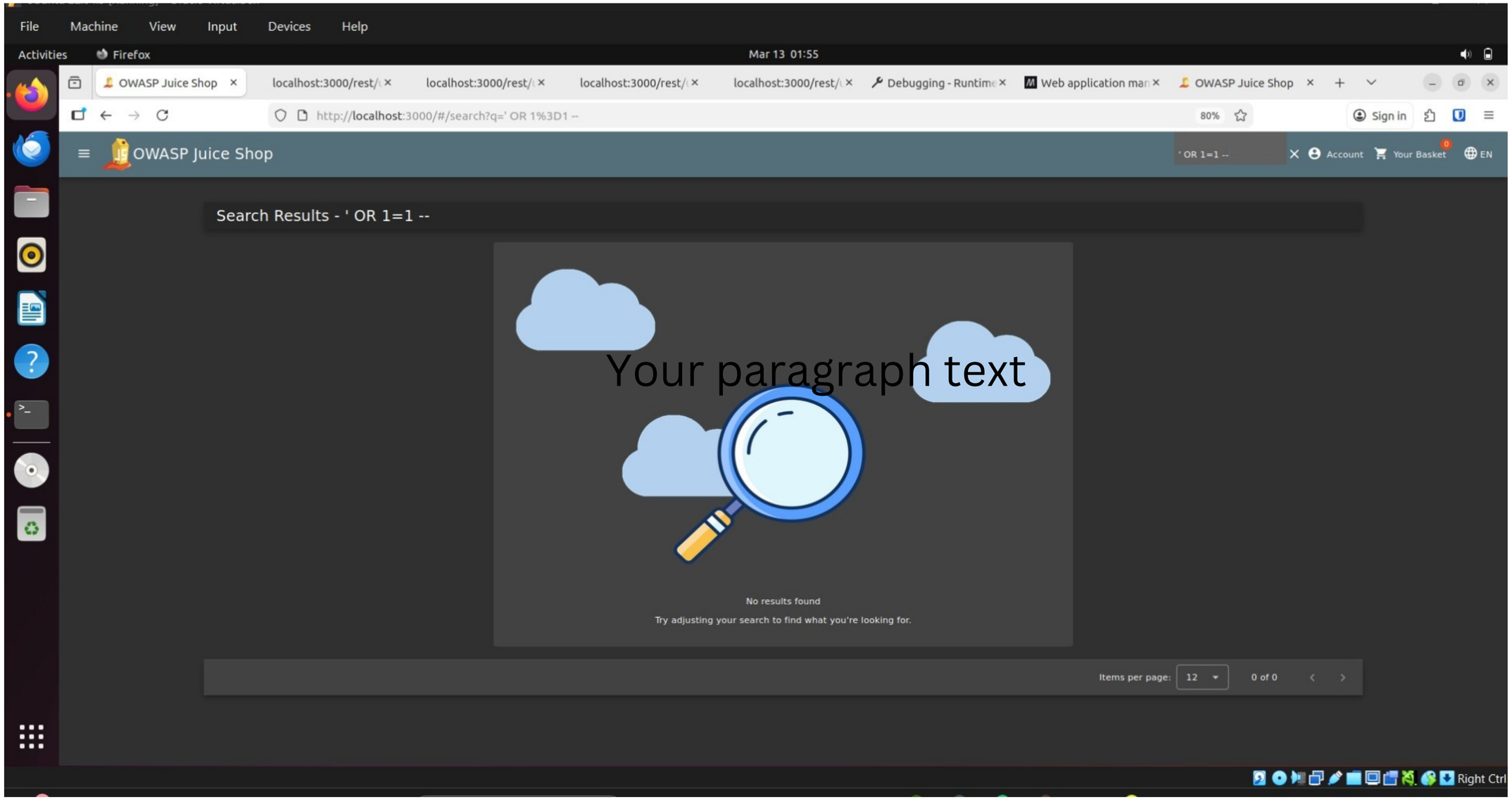
# 3: User Activity Logging – Authorized Actions




Actions performed by authenticated users such as viewing orders or adding products are logged. Monitoring these activities helps identify abnormal behavior.

# Attack 1 – SQL Injection Testing (Input Validation)

## Attack Demonstration – Input Validation Testing




Injection-style input tested in the search functionality to evaluate input validation behavior.
"The application successfully handled malicious input without exploitation."

The product search functionality was tested using specially crafted input commonly associated with injection attacks. The application processed the input without exposing database errors or returning unintended results. This behavior indicates that the system properly sanitizes user input and prevents injection-based exploitation through the search feature.

## What This Shows in Security Terms

| Test | Result |
| --- | --- |
| Injection input | Accepted but sanitized |
| Database error | ❌ Not exposed |
| Unexpected data | ❌ Not returned |
| Security posture | ✔ Proper input validation |

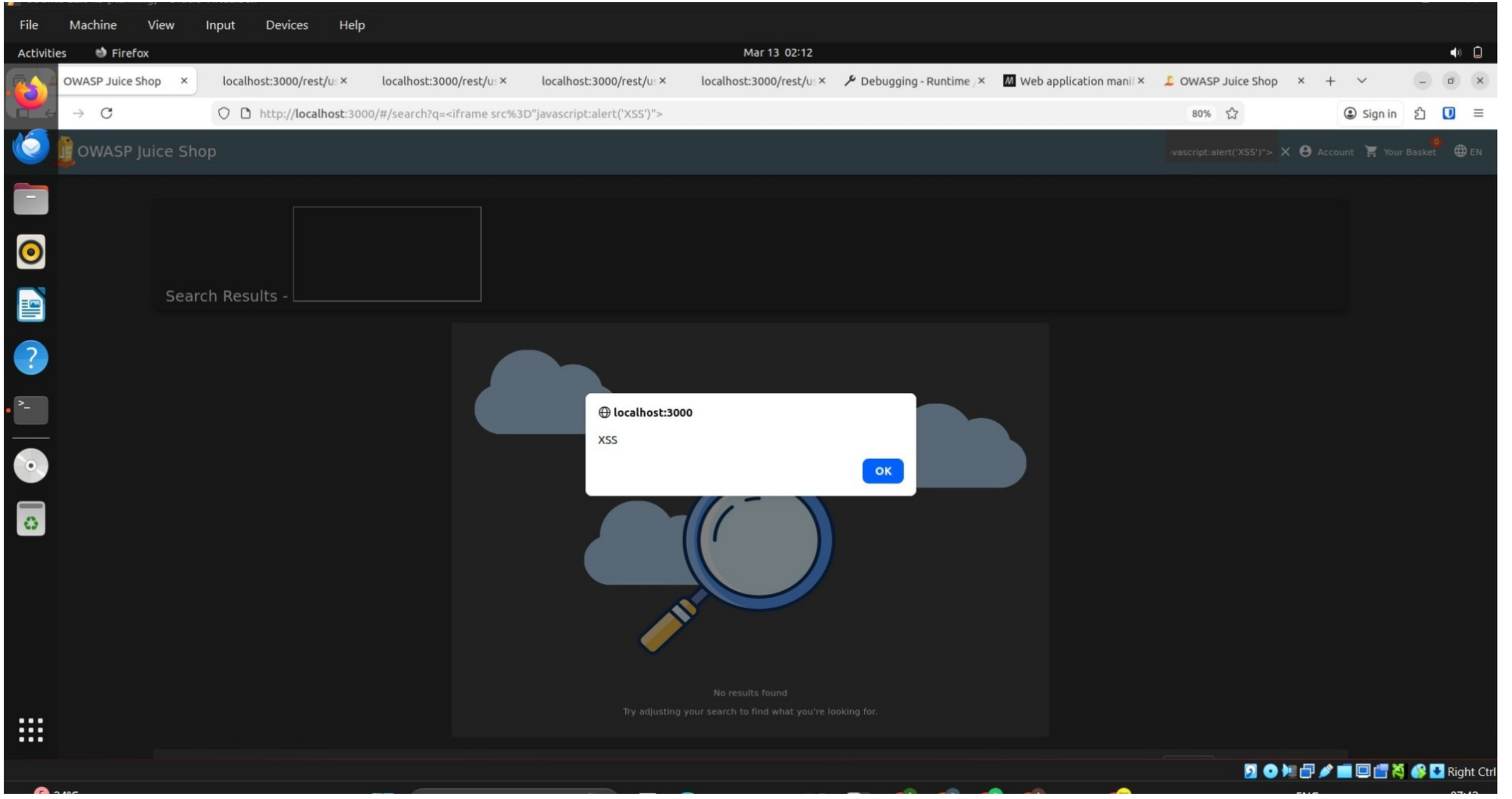
# Attack 2 – Cross-Site Scripting (XSS)

```
Attack Demonstration – Cross-Site Scripting (XSS)
```

## What XSS Means (Short Explanation)

You will add this text under the slide later:

> Cross-Site Scripting (XSS) occurs when an application improperly handles user input and allows malicious scripts to execute within the browser. Attackers may exploit XSS vulnerabilities to execute unauthorized scripts, steal session information, or manipulate webpage content.




Successful execution of injected JavaScript through the search functionality demonstrating a Cross-Site Scripting vulnerability.

During testing, the application search functionality was evaluated for potential Cross-Site Scripting vulnerabilities. A specially crafted input containing an embedded JavaScript payload was submitted through the search field. The application failed to properly sanitize the input, resulting in the execution of the injected script within the browser. This behavior demonstrates a Cross-Site Scripting vulnerability that could allow attackers to execute malicious scripts in the context of the user's browser.

**Potential Impact**

- Session hijacking
- Cookie theft
- Malicious script execution
- Defacement of web pages

**Recommended Mitigation**

- Implement strict input validation
- Use output encoding
- Apply Content Security Policy (CSP)
- Sanitize user-supplied input

# Attack Demonstration 3

## Access Control Testing (Administrative Interface)

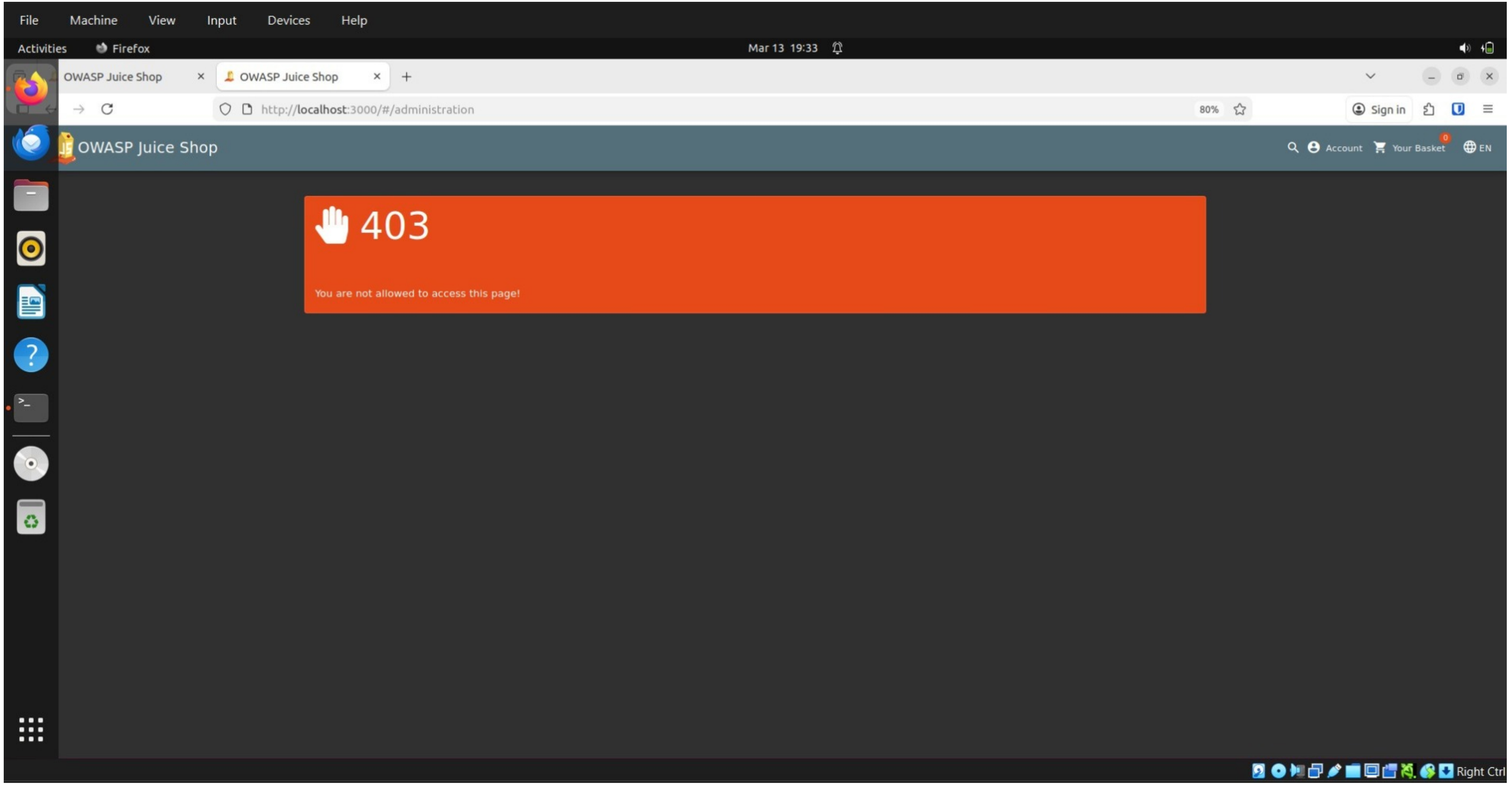
# 3: Tokenization (Conceptual – NO TOOL)

> **Payment Tokenization:**
> Tokenization replaces sensitive payment card information with a non-sensitive token that has no exploitable value. This ensures that actual card data is never stored or processed directly by the application, reducing the risk of data exposure during a breach.

## 4: PCI-DSS Compliance (Documentation)

> **PCI DSS Compliance Considerations:**
> The payment system follows PCI DSS best practices by ensuring encrypted data transmission, restricting access to cardholder data, avoiding storage of sensitive authentication data, and enforcing secure coding practices. These controls help protect cardholder information and reduce the risk of payment data compromise.

# Attack Demonstration – Access Control Testing




Unauthorized attempt to access the administrative interface resulting in a 403 Forbidden response.
“Attempting direct access to /administration endpoint without authentication

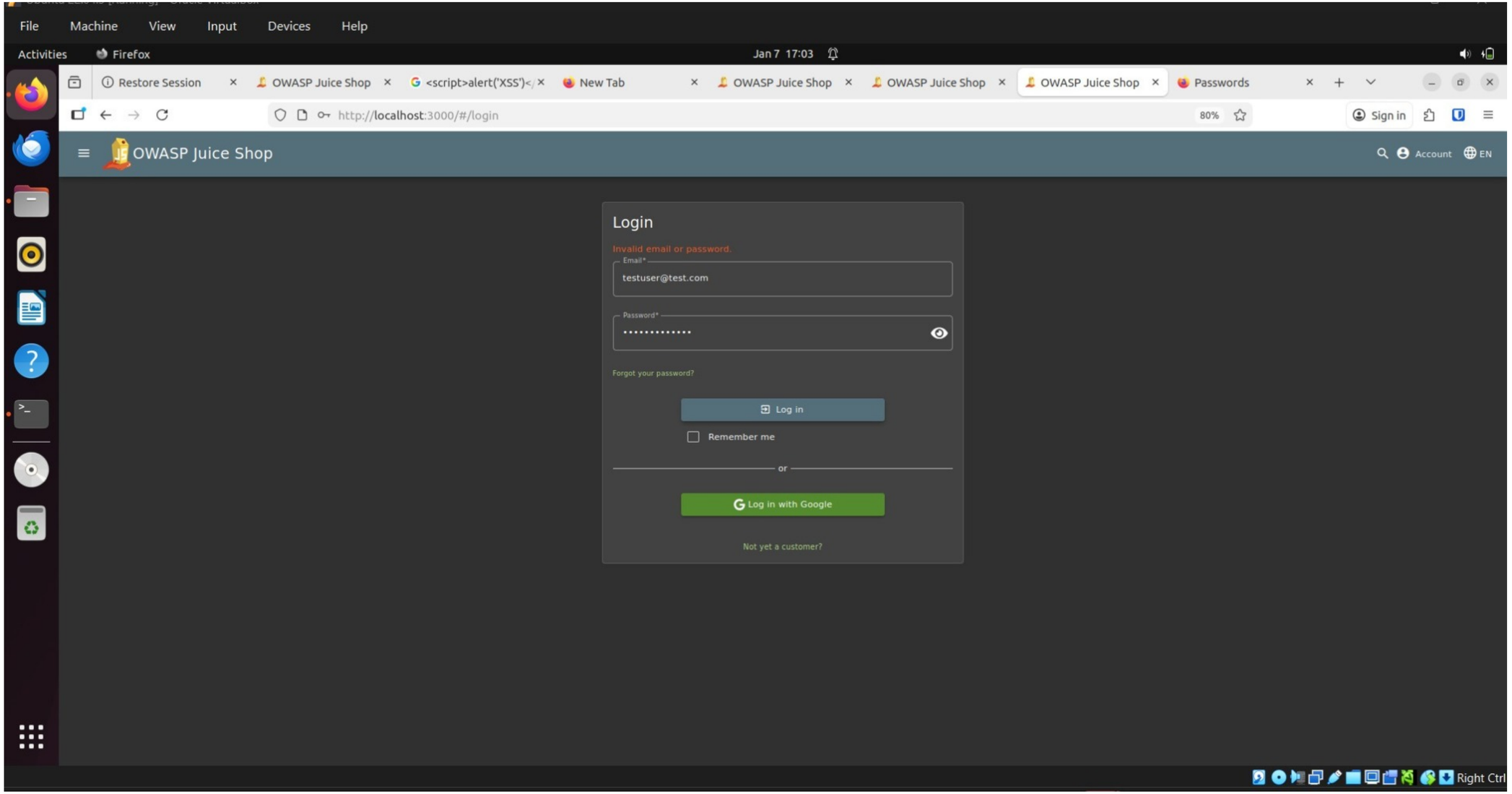
## What This Screenshot Demonstrates

| Test | Result |
| --- | --- |
| Unauthorized admin access attempt | ✅ Blocked |
| Server response | **403 Forbidden** |
| Security control | **Successfully enforced** |

This proves the system has **authorization protection for privileged resources**.

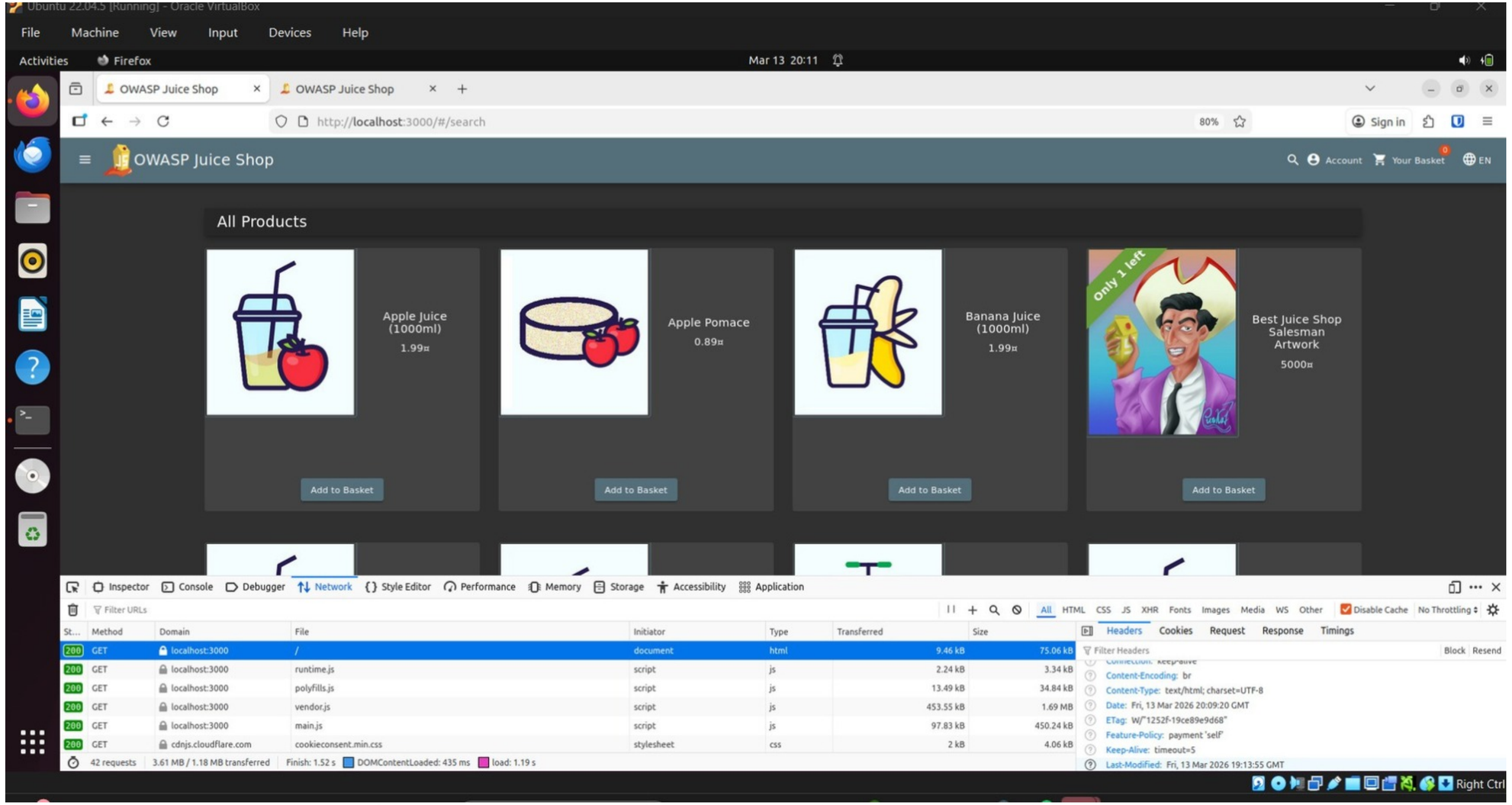
During security testing, an attempt was made to directly access the administrative interface of the application without proper authentication or authorization privileges. The application responded with a **403 Forbidden error**, indicating that access control mechanisms are in place to prevent unauthorized users from accessing sensitive administrative functionality. This demonstrates that the system correctly enforces role-based authorization policies to protect privileged resources.

# 5.0: Invalid Login Attempt




Failed authentication attempts are logged as security events. Monitoring such events helps detect brute-force attacks and unauthorized access attempts.

# Security Configuration Analysis – HTTP Response Headers




Inspection of HTTP response headers using browser developer tools to analyze the security configuration of the OWASP Juice Shop application.

During security testing, HTTP response headers were inspected using the browser developer tools to analyze the security configuration of the application. The analysis identified several headers including Content-Type, ETag, X-Content-Type-Options, and X-Frame-Options. These headers help protect the application against certain browser-based attacks such as MIME-type sniffing and clickjacking. Security header analysis is an important step in web application security assessments as it helps determine whether recommended security configurations are implemented.

| Security Header | Purpose |
| --- | --- |
| Content-Type | Defines the content type returned by the server |
| ETag | Helps with cache validation |
| X-Content-Type-Options | Prevents MIME type sniffing attacks |
| X-Frame-Options | Protects against clickjacking attacks |
| Content-Security-Policy | Helps prevent XSS attacks by restricting allowed content sources |

# 🛡️5:INCIDENT RESPONSE

Logged security events enabletimely incidentdetection and response. In case of suspicious activity, administrators can analyze logs, identify affected accounts, revoke sessions, and apply corrective controls such as password resets or account lockouts.

# PHASE 6 — Security Hardening, Phishing Protection, Monitoring & Incident Response

# Secure Coding and Website Hardening

- input validation and sanitization
- secure coding practices

Based on the security testing performed in earlier phases, the application should adopt secure coding practices to reduce the risk of common web vulnerabilities such as SQL injection, Cross-Site Scripting (XSS), and unauthorized access. Secure coding improves the resilience of the e-commerce platform by ensuring that user input is validated, output is encoded, and sensitive logic is handled securely on the server side.

- Validate and sanitize all user input
- Use server-side validation for form fields
- Apply output encoding to prevent XSS
- Avoid hardcoded credentials and secrets
- Use secure session management
- Follow least privilege for application component

parameterized queries (prepared statements)

# Content Security Policy (CSP) and Web Application Firewall (WAF)

- “implement CSP” → **“Implement CSP (Content Security Policy)”**
- “deploy WAF” → **“Deploy Web Application Firewall (WAF)”**

To strengthen website security, the application should implement a Content Security Policy (CSP) and deploy a Web Application Firewall (WAF). CSP helps restrict which scripts and resources can be loaded by the browser, reducing the risk of Cross-Site Scripting attacks. A WAF provides an additional layer of defense by filtering malicious HTTP traffic and blocking suspicious requests before they reach the application.

- Implement CSP to allow only trusted resources
- Restrict inline JavaScript execution
- Use WAF rules to detect malicious requests
- Block repeated suspicious payloads
- Monitor unusual traffic patterns

**Example CSP**

```
Content-Security-Policy:  default-src 'self'; script-src 'self';
```

**Example WAF role**

```
Detect and block malicious payloads such as XSS and injection attempts.
```

# Phishing Protection Strategy

- SPF
- DKIM
- DMARC
- user education

Phishing protection is essential for protecting customer accounts and preventing email-based impersonation attacks. The e-commerce platform should implement email authentication standards such as SPF, DKIM, and DMARC to verify legitimate email sources and reduce spoofing. In addition, users should be educated to identify suspicious emails, fake links, and fraudulent login pages.

- Implement SPF to define authorized mail servers
- Use DKIM to digitally sign outgoing emails
- Configure DMARC to enforce email authentication policy
- **Train users to identify and avoid suspicious links"**
- Encourage users to verify domain names before login

| Protocol | Purpose |
| --- | --- |
| SPF | Verifies sending mail servers |
| DKIM | Adds email signature validation |
| DMARC | Enforces email authentication policy |

**user awareness points**

**5 signs of phishing**

1. Suspicious sender address
2. Urgent or threatening language
3. Fake login links
4. Unexpected attachments
5. Spelling or domain mismatches

"Enable logging and monitoring for suspicious activities"

# Slide 5 — Real-Time Monitoring Strategy

## Real-Time Monitoring Strategy

- “Implement real-time monitoring system”
- detect suspicious activities

Real-time monitoring helps detect suspicious behavior, unauthorized access attempts, and abnormal system activity before they lead to major security incidents. Monitoring tools should track authentication events, access control violations, repeated failed login attempts, unusual traffic patterns, and application errors. Continuous monitoring improves visibility into the security posture of the e-commerce platform.

## Monitoring Capabilities

- Monitor failed login attempts
- Detect repeated access violations
- Track unusual API or traffic behavior
- Log authentication and session events
- Alert administrators on critical anomalies

| Tool | Purpose |
| --- | --- |
| Wazuh | Security monitoring and alerting |
| ELK Stack | Log collection and visualization |
| Fail2Ban | Blocks repeated malicious access attempts |

# Slide 6 — Incident Response Plan

## Incident Response Plan

**This phase includes**

- incident response plan
- simulation / response strategy

An incident response plan defines how the organization should respond when a security incident occurs. A structured process helps reduce damage, restore services quickly, and preserve evidence for investigation. In an e-commerce environment, incident response is essential for handling account compromise, payment fraud, data exposure, and web application attacks.

1. **Detect** – identify suspicious activity or security alerts
2. **Contain** – isolate affected systems or user accounts
3. **Eradicate** – remove malicious code or unauthorized access
4. **Recover** – restore normal operations securely
5. **Report** – document findings and notify stakeholders
6. **Improve** – apply lessons learned and strengthen controls

**Example Incident:**

Unauthorized admin access attempt detected

**Response:**

Block IP, review logs, verify account compromise, rotate credentials, document incident

# PHASE 7 — Security Audit and Penetration Testing

## Section 1 — Security Audit

A security audit was conducted to evaluate the security posture of the e-commerce application. The audit focused on verifying the implementation of key security controls such as authentication mechanisms, encryption practices, access control policies, logging, and protection against common web vulnerabilities. The objective was to ensure that the platform follows industry-recommended security standards and best practices. **based on OWASP Top 10 guidelines**

## Section 2 — Penetration Testing

# Penetration Testing

Penetration testing was performed to simulate real-world cyber attacks against the application. Various attack techniques were simulated to identify exploitable vulnerabilities and evaluate how effectively the system defends against malicious activities. The testing included injection attacks, authentication bypass attempts, session analysis, and security header validation.

## Section 3 — Testing Activities Table

| Testing Activity | Tool / Method Used |
| --- | --- |
| Vulnerability scanning | OWASP ZAP |
| SQL Injection testing | OWASP ZAP |
| Cross-Site Scripting (XSS) testing | OWASP ZAP |
| Authentication & RBAC testing | Manual testing |
| Security header analysis | Browser DevTools |
| Session and access control testing | Manual inspection |

'Authentication bypass testing → Manual testing'

# Section 4 — Key Findings

## Key Findings

**Findings Summary**

- Several security mechanisms such as authentication controls and role-based access restrictions were successfully tested.
- Security headers were reviewed to evaluate browser-level protections.
- Potential vulnerabilities such as injection and cross-site scripting were analyzed using automated and manual testing techniques.
- The testing results help identify areas for improvement and guide the implementation of stronger security controls.

# PHASE 8 — Data Privacy & Compliance

Protecting Customer Information and Ensuring Regulatory Compliance

Slide 2 — Data Privacy Policies

# Data Privacy Policy

During the security evaluation of the e-commerce platform, data privacy policies were reviewed to ensure that customer information is handled securely and responsibly. The platform implements policies that define how user data is collected, stored, processed, and protected. These policies emphasize transparency, secure storage of personal information, limited access to sensitive data, and adherence to industry best practices for protecting user privacy.

Bullet Points:

- Collection of only necessary customer information
- Secure storage of user data and payment details
- Restricted access to sensitive customer data
- **Prevention of unauthorized data disclosure”**
- Transparency regarding how user information is used

# Slide 3 — Regulatory Compliance

## Regulatory Compliance (GDPR & CCPA)

To ensure responsible data handling, the e-commerce platform aligns with major data protection regulations such as the General Data Protection Regulation (GDPR) and the California Consumer Privacy Act (CCPA). These regulations establish strict guidelines for protecting personal data and ensuring user rights related to data access, correction, and deletion. Implementing these compliance standards helps strengthen customer trust and enhances the overall security posture of the platform.

*"Compliance ensures legal adherence and reduces regulatory risk."*

| Compliance Area | Description |
| --- | --- |
| Data Protection | Personal data stored securely |
| User Consent | Users informed about data collection |
| Data Access Rights | Users can request access to their data |
| Data Deletion | Users can request deletion of personal information |
| Transparency | Clear privacy policies available |

# Project Outcome & Learning

## Project Outcomes

- Successfully deployed and tested the OWASP Juice Shop vulnerable web application.
- Identified and demonstrated real-world web vulnerabilities such as SQL Injection and Cross-Site Scripting (XSS).
- Analyzed authentication and authorization weaknesses through login and session handling tests.
- Evaluated insecure payment workflow risks and the importance of secure communication channels.
- Implemented and reviewed security logging, monitoring, and incident response mechanisms.
- Mapped vulnerabilities and mitigations to OWASP Top 10 security risks.
- Documented practical evidence through screenshots and phase-wise explanations.

## Key Learnings

- Gained hands-on experience in identifying and exploiting web application vulnerabilities.
- Learned how attackers misuse input fields, authentication logic, and sessions.
- Understood the importance of secure coding practices such as input validation and output encoding.
- Learned how HTTPS, secure cookies, and proper session handling protect user data.
- Gained practical knowledge of security logging, monitoring, and incident response processes.
- Developed skills in vulnerability documentation and security reporting.
- Improved understanding of secure application design and defensive security strategies.

This project significantly enhanced practical cybersecurity skills and provided real-world exposure to web application security assessment and mitigation techniques.

# References


1. OWASP Foundation. (n.d.). OWASP Juice Shop. https://owasp.org/www-project-juice-shop/

2. OWASP Foundation. (2021). OWASP Top 10:2021. https://owasp.org/Top10/2021/

3. OWASP Foundation. (n.d.). OWASP Web Security Testing Guide. https://owasp.org/www-project-web-security-testing-guide/

4. ZAP Development Team. (n.d.). ZAP Documentation. https://www.zaproxy.org/docs/

5. OWASP Foundation. (n.d.). Content Security Policy Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html

# Questions?

I hope my work and effort will meet your expectations. Thank you for giving me this opportunity.”

# Thank you!